ccomp
Deep Dive into the C Compilation Process: From Source Code to Machine Execution

Project: ccomp
GitHub: https://github.com/lrisguan/ccomp
Author: ccomp contributors

2026-02-23

Contents

Chapter 1

How Computers Execute Programs

1.1 Introduction

Before diving into the intricacies of C compilation, we need to understand what happens *after* compilation—when your program actually runs on a physical processor. This chapter establishes the fundamental execution model of modern computers: how CPUs execute instructions, how memory is organized, and how function calls work at the hardware level.

Why start here? Because understanding the target execution model explains **why** compilers make the decisions they do. Every optimization, every register allocation, and every function call convention exists to serve the underlying hardware architecture.

1.2 The CPU Execution Model

1.2.1 What is a CPU?

A Central Processing Unit (CPU) is a state machine that repeatedly fetches, decodes, and executes instructions. At its core, a CPU has:

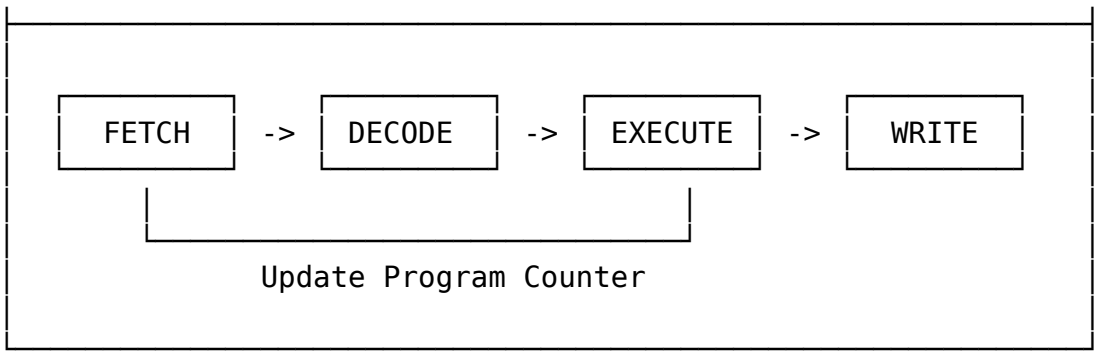
1. **Registers**: Fast storage locations within the CPU
2. **Arithmetic Logic Unit (ALU)**: Performs mathematical and logical operations
3. **Control Unit**: Orchestrates the instruction cycle
4. **Buses**: Connect the CPU to memory and other components

The CPU maintains a piece of architectural state that gets updated with each instruction execution. Think of this state as the CPU's "working memory"—everything it needs to know about the current program's execution.

1.2.2 The Instruction Cycle

The CPU operates through a continuous loop called the **instruction cycle** (also known as the fetch-decode-execute cycle):

```
┌──────────────────────────────────────────────────────────────┐
│                    Instruction Cycle                         │
```



Let's break down each stage:

Fetch

The CPU reads the next instruction from memory at the address stored in the **Program Counter (PC)** register (also called the Instruction Pointer or IP). After fetching, the PC is incremented to point to the next instruction.

Decode

The CPU interprets the fetched bits to determine:

- What operation to perform (add, subtract, jump, etc.)
- Which registers to use
- What memory addresses to access (if any)

Execute

The CPU actually performs the operation:

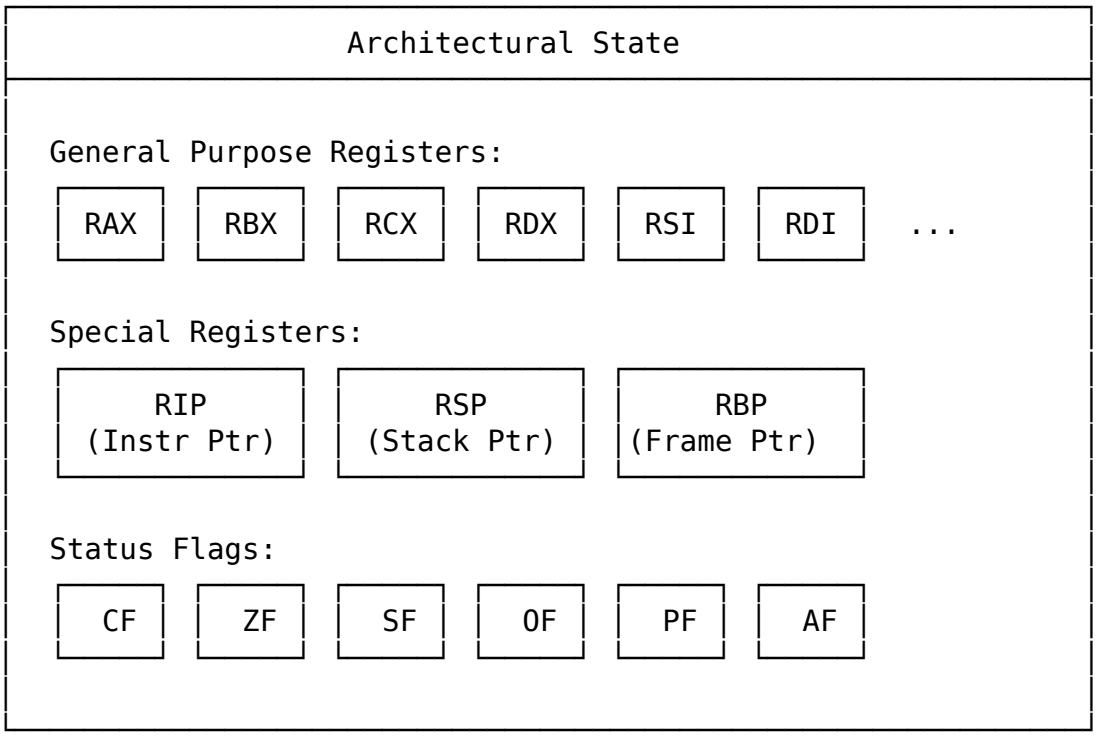
- Arithmetic calculations in the ALU
- Memory reads or writes
- Conditional checks that affect control flow

Writeback (optional)

Many instructions write results back to registers or memory, updating the architectural state.

1.2.3 Architectural State

The **architectural state** is the set of all registers and flags that define the current execution context:



This state is crucial because:

- **Function calls** preserve parts of this state
- **Context switches** save and restore this state
- **Debuggers** inspect this state to show program behavior

1.3 Registers: The CPU's Working Memory

Registers are the fastest storage available in a computer. They're built directly into the CPU hardware and can be accessed in a single clock cycle. Modern CPUs have two main categories of registers:

1.3.1 General-Purpose Registers (GPRs)

Despite the name, these registers aren't truly "general purpose"—conventions and hardware features give each register specific roles.

On x86-64 (the architecture used by most modern desktop and server computers):

Register	Name	Special Use
RAX	Accumulator	Return values for functions
RBX	Base	Callee-saved register
RCX	Counter	Used for loops and string operations
RDX	Data	Used for multiplication/division
RSI	Source Index	Source for string operations
RDI	Destination Index	Destination for string operations
RSP	Stack Pointer	Points to top of stack
RBP	Base Pointer	Points to current stack frame
R8-R15	Extended registers	Additional general-purpose registers

Warning

Historical Note: The "R" prefix denotes 64-bit registers in x86-64. Originally, these were 16-bit registers (AX, BX, etc.), then extended to 32-bit (EAX, EBX), and finally to 64-bit (RAX, RBX). You can still access the lower portions of these registers: RAX contains EAX, which contains AX, which contains AH (high byte) and AL (low byte).

1.3.2 Special-Purpose Registers

Some registers have dedicated functions:

Program Counter (RIP on x86-64)

- Holds the memory address of the *next* instruction to execute
- Automatically updated after most instructions
- Can be explicitly modified by jump, call, and return instructions

Stack Pointer (RSP on x86-64)

- Points to the top of the stack (specifically, the lowest address in the current stack frame)
- Grows downward (toward lower addresses) on most architectures
- Crucial for function calls and local variable storage

Flags Register (RFLAGS)

- Contains individual bits (flags) set by arithmetic operations
- Key flags include:
  - **ZF (Zero Flag)**: Set if the result was zero
  - **SF (Sign Flag)**: Set if the result was negative (in two's complement)
  - **CF (Carry Flag)**: Set if an operation overflowed its destination
  - **OF (Overflow Flag)**: Set if signed overflow occurred

These flags enable conditional branching (like `if` statements and loops).

1.3.3 Caller-Saved vs. Callee-Saved Registers

One of the most important conventions in modern programming is the division of registers into **caller-saved** and **callee-saved** registers. This convention makes function calls efficient.

Caller-saved registers (also called volatile registers):

- Can be overwritten by a called function
- If the caller needs to preserve a value, it must save it before calling
- On x86-64: RAX, RCX, RDX, RSI, RDI, R8-R11

Callee-saved registers (also called non-volatile registers):

- Must be preserved by functions that modify them
- If a function wants to use these registers, it must save the original value and restore it before returning
- On x86-64: RBX, RBP, R12-R15

Warning

Why this matters: This convention is part of the **calling convention** or **ABI** (Application Binary Interface). When a C compiler generates code, it knows which registers it can use freely and which it must preserve. Chapter 9 covers ABIs in detail.

1.4 The Memory Model

1.4.1 Memory as a Giant Byte Array

From the CPU's perspective, memory is a contiguous array of bytes, each with a unique address. On a 64-bit system, addresses range from 0 to $2^{64} - 1$ (though in practice, only about 48 bits are used currently).

```
Address  Contents
┌──────────────────────────────────────────────┐
│ 0x0000...0000  │ ?                           │
│ 0x0000...0001  │ ?                           │
│ 0x0000...0002  │ ?                           │
│ 0x0000...0003  │ ?                           │
│     ...        │                             │
│ 0x7FFF...0000  │ (often stack start)         │
│     ...        │                             │
└──────────────────────────────────────────────┘
```

Key properties:

1. **Byte-addressability**: Each byte has a unique address

2. **Word size**: The CPU typically accesses memory in chunks (e.g., 4 bytes at a time on a 32-bit system, 8 bytes on 64-bit)
3. **Alignment**: Accessing multi-byte values from addresses that are multiples of their size is faster (and sometimes required)

1.4.2 Endianness

When a multi-byte value is stored in memory, there's a choice: which byte goes at the lowest address? This is called **endianness**.

Little-endian (used by x86 and x86-64):

- The least significant byte is stored at the lowest address
- Example: 0x12345678 stored as `78 56 34 12`

```
Address:   0x1000  0x1001  0x1002  0x1003
Content:    0x78    0x56    0x34    0x12
                             ↑
                        Least significant byte
```

Big-endian (used by some network protocols and older architectures):

- The most significant byte is stored at the lowest address
- Example: 0x12345678 stored as `12 34 56 78`

Warning

Practical Impact: Endianness matters when:

- Reading binary files created on different architectures
- Implementing network protocols (network byte order is big-endian)
- Debugging memory dumps

1.4.3 Virtual Memory

Modern operating systems use **virtual memory**, which gives each process the illusion of having its own private address space. The CPU's Memory Management Unit (MMU) translates virtual addresses to physical addresses transparently.

Benefits:

- **Isolation**: Processes can't access each other's memory
- **Protection**: Pages can be marked read-only or execute-only
- **Efficiency**: Only loaded portions of a program need to be in physical memory
- **Simplicity**: Each program can use the same address range (e.g., stack always starting at a high address)

We'll explore virtual memory in depth in Chapter 10.

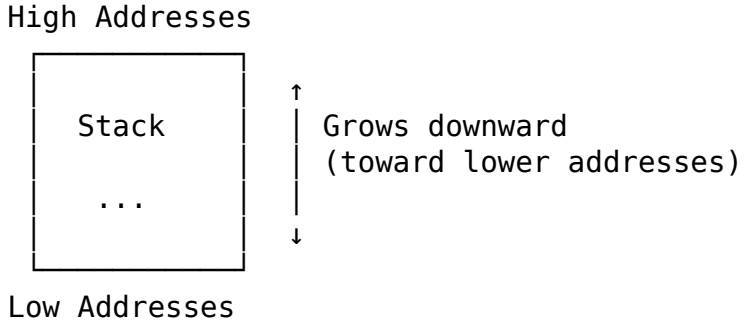
1.5 The Stack and Function Calls

1.5.1 What is the Stack?

The **stack** is a region of memory used for:

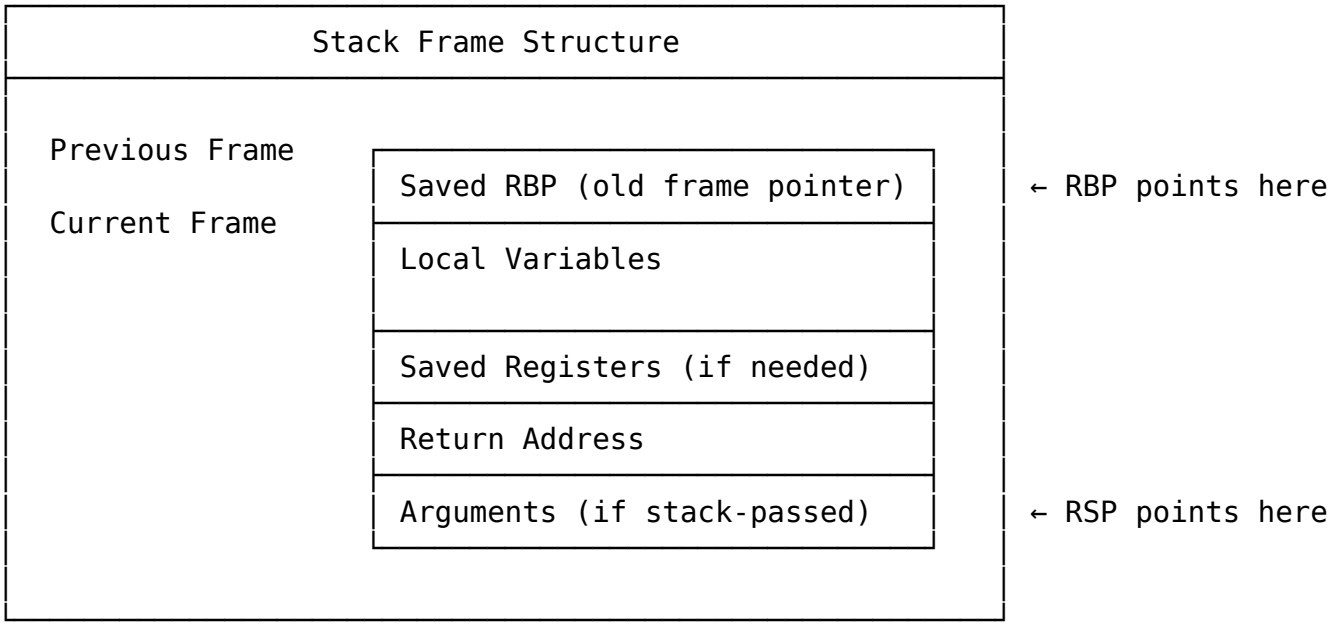
- Function call frames
- Local variables
- Return addresses
- Temporary storage

The stack grows **downward** (toward lower addresses), and the stack pointer (RSP) always points to the *lowest* used address (the "top" of the stack).



1.5.2 Stack Frames

Each function call creates a **stack frame** (also called an activation record). The frame contains:



The frame pointer (RBP) serves as a stable reference point within a function's stack frame. While some optimizations omit the frame pointer (using RSP as a general-purpose register instead), it's extremely useful for debugging.

1.5.3 Function Prologue and Epilogue

Every function has a standard structure:

Prologue (executed when entering the function):

```
push rbp ; Save caller's frame pointer
mov rbp, rsp ; Establish new frame pointer
sub rsp, N ; Allocate space for local variables (N bytes)
```

Epilogue (executed when exiting the function):

```
mov rsp, rbp ; Deallocate local variables (restore stack pointer)
pop rbp ; Restore caller's frame pointer
ret ; Return to caller (uses saved return address)
```

Let's trace what happens with this C code:

```
int add(int a, int b) {
   int result = a + b;
   return result;
}

int main(void) {
   return add(5, 3);
}
```

When `main` calls `add`:

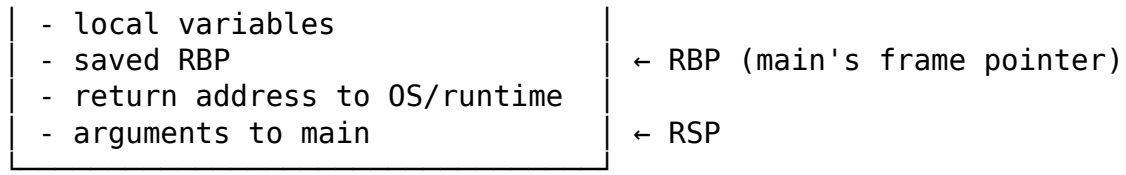
1. **Before call**: Arguments are prepared (in registers or on stack)
2. **Call instruction**: The `call` instruction pushes the **return address** onto the stack and jumps to `add`
3. **Prologue in `add`**: Sets up the new stack frame
4. **Function body**: Executes the addition
5. **Epilogue in `add`**: Tears down the stack frame
6. **Return**: The `ret` instruction pops the return address and resumes execution in `main`

1.5.4 Detailed Stack Growth Example

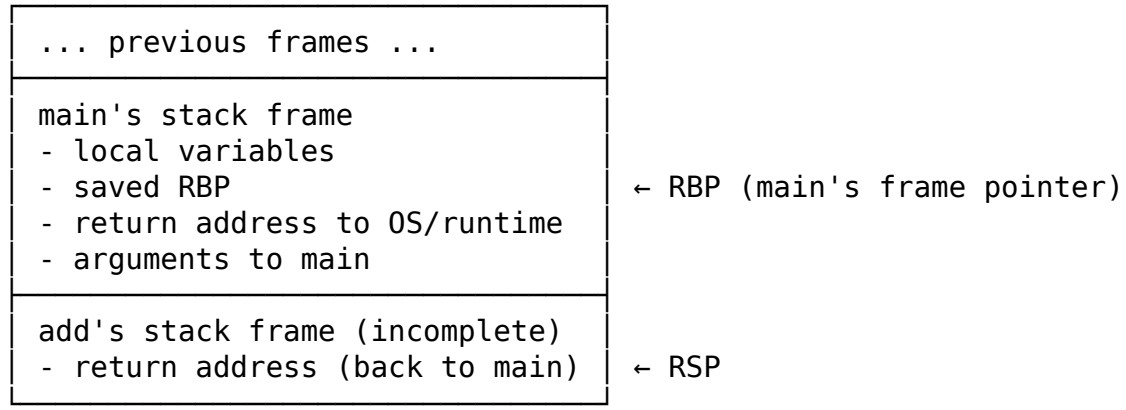
Let's visualize the complete stack behavior:

`State before calling add(5, 3):`

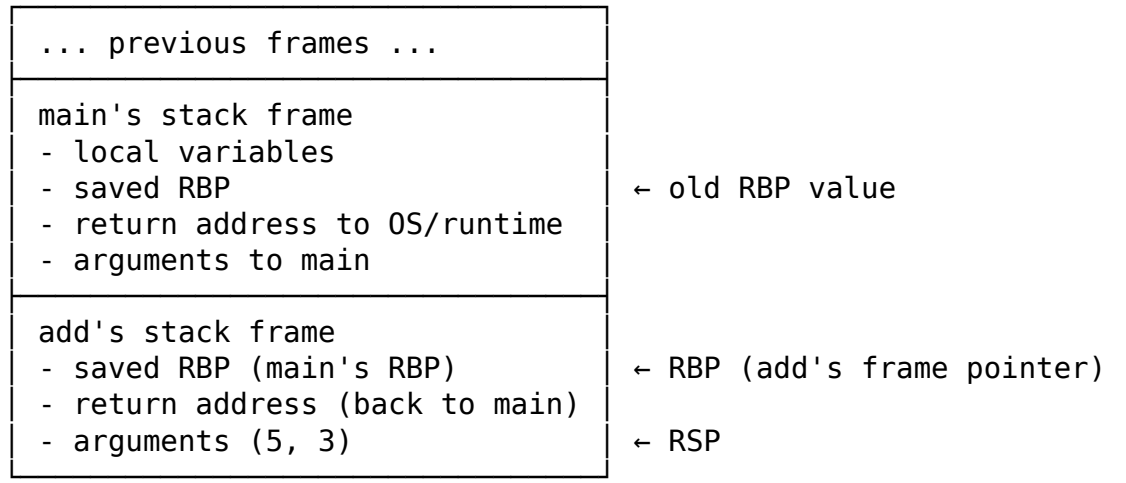
```
┌──────────────────────────────────┐
│ ... previous frames ...          │
├──────────────────────────────────┤
│ main's stack frame               │
```



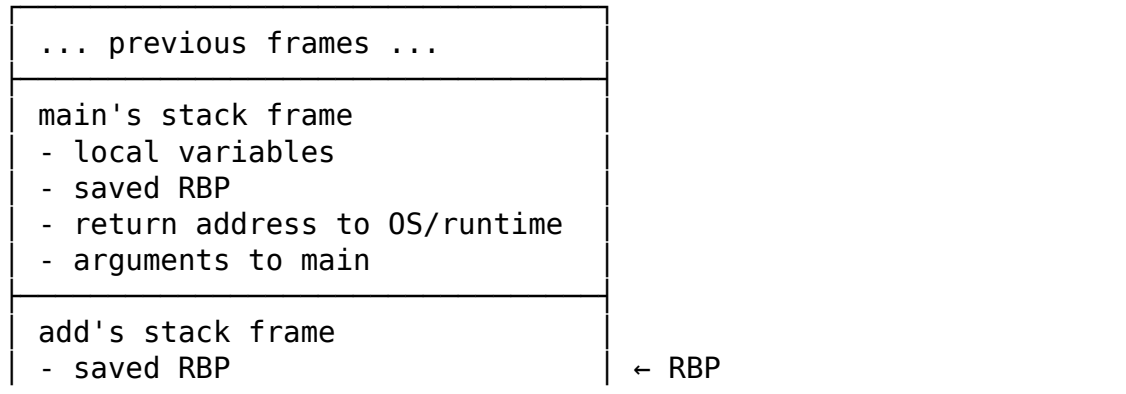
State after call to add (before prologue):



State after add's prologue:



State after allocating local variables:



```
| - return address                  |
| - arguments (5, 3)                |
| - local var: result (4 bytes)     | ← RSP
└───────────────────────────────────┘
```

1.6 Calling Conventions

1.6.1 What is a Calling Convention?

A **calling convention** is a set of rules that defines how functions call each other. It specifies:

1. **Argument passing**: Where to put function arguments (registers vs. stack)
2. **Return values**: Where to place return values
3. **Stack cleanup**: Who cleans up arguments from the stack (caller or callee)
4. **Register preservation**: Which registers must be preserved across calls
5. **Name mangling** (in some languages): How to encode function names with type information

Definition

Critical Insight: Without calling conventions, code compiled by different compilers couldn't work together. The calling convention is the **contract** that allows separate compilation to work.

1.6.2 The System V AMD64 ABI (Linux/Unix)

The most common calling convention for x86-64 on Unix-like systems is the **System V AMD64 ABI**. Here's how it works:

Argument Passing:

- First 6 integer/pointer arguments: RDI, RSI, RDX, RCX, R8, R9 (in order)
- First 8 floating-point arguments: XMM0-XMM7
- Additional arguments: Passed on the stack

Return Values:

- Integer/pointer returns: RAX
- Floating-point returns: XMM0
- Large structs: Hidden pointer parameter (address where to store result)

Register Preservation:

- Callee-saved: RBX, RBP, R12-R15

- Caller-saved: RAX, RCX, RDX, RSI, RDI, R8-R11

Stack Alignment:

- The stack must be 16-byte aligned before any `call` instruction

Let's see an example:

```
long compute(long a, long b, long c, long d, long e, long f, long g) {
   return a + b + c + d + e + f + g;
}

int main(void) {
   return (int)compute(1, 2, 3, 4, 5, 6, 7);
}
```

The arguments would be passed as follows:

- `a` (1) → RDI
- `b` (2) → RSI
- `c` (3) → RDX
- `d` (4) → RCX
- `e` (5) → R8
- `f` (6) → R9
- `g` (7) → On the stack

1.6.3 The Microsoft x64 Calling Convention (Windows)

Windows uses a different calling convention:

Argument Passing:

- First 4 arguments: RCX, RDX, R8, R9 (in order)
- Additional arguments: Passed on the stack
- For floating-point: XMM0-XMM3 for first 4 arguments

Key Differences from System V:

- Different registers for arguments
- Stack space is always allocated for register arguments (shadow space)
- Different register preservation rules

Warning

Practical Note: This is why you can't always link object files between Windows and Linux, even on the same architecture. The calling conventions are incompatible.

1.7 Control Flow and Branch Prediction

1.7.1 Conditional Jumps

The CPU executes instructions sequentially, but conditional jumps (branches) allow programs to make decisions. Conditional jumps depend on the flags register:

```
cmp rax, rbx ; Compare RAX and RBX (sets flags)
je target ; Jump if equal (ZF == 1)
jne target ; Jump if not equal (ZF == 0)
jg target ; Jump if greater (signed comparison)
jl target ; Jump if less (signed comparison)
ja target ; Jump if above (unsigned comparison)
jb target ; Jump if below (unsigned comparison)
```

Modern CPUs use **branch prediction** to guess which way a branch will go and speculatively execute those instructions. Correct predictions improve performance; mispredictions cause the pipeline to be flushed.

1.7.2 Loops

Loops are implemented using conditional jumps:

```
for (int i = 0; i < 10; i++) {
   // loop body
}
```

Might compile to:

```
   xor eax, eax ; i = 0
.loop:
   cmp eax, 10 ; compare i with 10
   jge .end ; if i >= 10, exit loop
   ; loop body here
   inc eax ; i++
   jmp .loop ; repeat
.end:
```

1.8 Putting It All Together: A Complete Example

Let's trace through a complete program execution to see all these concepts in action:

```
int multiply_by_two(int x) {
   return x * 2;
}

int add_and_multiply(int a, int b) {
   int sum = a + b;
   return multiply_by_two(sum);
}

int main(void) {
   int result = add_and_multiply(5, 7);
```

```
    return result == 24 ? 0 : 1;
}
```

Here's what happens when this program runs:

1. **Program startup**: The OS loader sets up the initial stack and calls `main`
2. **In `main`**:
   - Arguments to `add_and_multiply` are prepared: RDI = 5, RSI = 7
   - The `call` instruction pushes the return address onto the stack
   - Execution jumps to `add_and_multiply`
3. **In `add_and_multiply`**:
   - Prologue: Saves RBP, sets up new frame
   - Adds RDI and RSI: `lea eax, [rdi + rsi]` (result in EAX)
   - Prepares argument to `multiply_by_two`: EDI = EAX (the sum)
   - The `call` instruction pushes return address, jumps to `multiply_by_two`
4. **In `multiply_by_two`**:
   - Prologue: Saves RBP, sets up new frame
   - Multiplies by 2: `lea eax, [rdi + rdi]` (efficient way to multiply by 2)
   - Epilogue: Restores RBP
   - Return: Pops return address, returns to `add_and_multiply`
5. **Back in `add_and_multiply`**:
   - The return value (24) is now in EAX
   - Epilogue: Restores RBP
   - Return: Returns to `main`
6. **Back in `main`**:
   - Compares EAX (24) with 24: `cmp eax, 24`
   - Sets flags accordingly (ZF will be set because they're equal)
   - Conditional move: `sete al` (AL = 1 if equal, 0 if not)
   - Zero-extends: `movzx eax, al`
   - Returns this value as the program's exit status

1.9 Common Pitfalls

1.9.1 Pitfall 1: Stack Overflow

Each function call consumes stack space. Deep recursion or large local arrays can exhaust the stack:

```
// Dangerous: Recursion with no base case
void infinite_recursion(void) {
   char buffer[1024]; // Each call uses 1KB+ of stack
   infinite_recursion();
}
```

When the stack is exhausted, the program crashes with a **segmentation fault**.

1.9.2 Pitfall 2: Misaligned Accesses

Accessing multi-byte values from misaligned addresses can hurt performance or cause crashes:

```
// Potentially slow or crashing on some architectures
void misaligned_access(void* p) {
   int* ip = (int*)((char*)p + 1); // Misaligned!
   *ip = 42; // May be slow or crash
}
```

1.9.3 Pitfall 3: Register Starvation

When a function needs more registers than available, the compiler must spill some to memory (the stack), which hurts performance:

```
// Uses many variables - may cause register spills
int complex_calc(int a, int b, int c, int d, int e,
                 int f, int g, int h, int i, int j) {
   int t1 = a + b;
   int t2 = c * d;
   int t3 = e - f;
   int t4 = g / h;
   int t5 = i << 2;
   return t1 + t2 + t3 + t4 + t5;
}
```

1.10 Key Takeaways

1. **The CPU is a state machine**: It fetches, decodes, and executes instructions in a continuous loop, updating architectural state with each instruction.

2. **Registers are fast but limited**: General-purpose registers provide the fastest storage, but there are only a handful. Callee-saved vs. caller-saved conventions enable efficient function calls.

3. **Memory is byte-addressable**: Each byte has a unique address, and multi-byte values are stored according to the system's endianness (little-endian on x86-64).

4. **The stack enables function calls**: Each function call creates a stack frame containing local variables, saved registers, and return addresses. The stack grows downward, managed by the stack pointer (RSP) and frame pointer (RBP).

5. **Calling conventions are contracts**: They define how functions call each other—argument passing, return values, stack cleanup, and register preservation. The System V AMD64 ABI is the standard on Linux/Unix x86-64 systems.

6. **Control flow uses jumps and flags**: Conditional jumps depend on status flags set by comparison and arithmetic operations, enabling `if` statements and loops.

7. **Understanding execution explains compilation**: The hardware execution model drives compiler decisions. Every optimization targets some aspect of the underlying architecture.

1.11 Looking Ahead

Now that we understand how programs execute at the hardware level, we can explore how C source code gets transformed into machine instructions. In Chapter 1, we'll examine the complete compilation pipeline—from source code to executable program.

Chapter 2

C Compilation Overview

2.1 Introduction

When you write a C program, you're creating human-readable source code. But computers don't execute C code directly—they execute machine instructions, binary patterns that the CPU understands. The **compilation pipeline** is the bridge between these two worlds, transforming your source code through a series of well-defined stages, each producing intermediate artifacts.

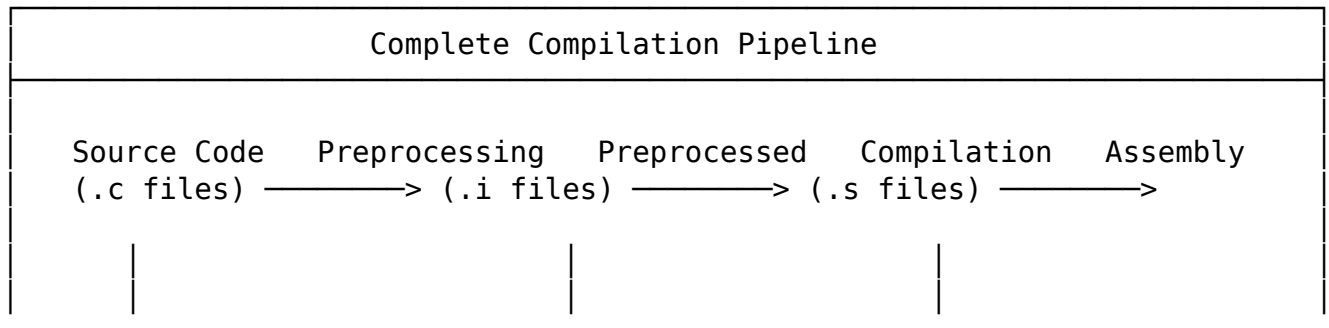
Understanding this pipeline is essential for:

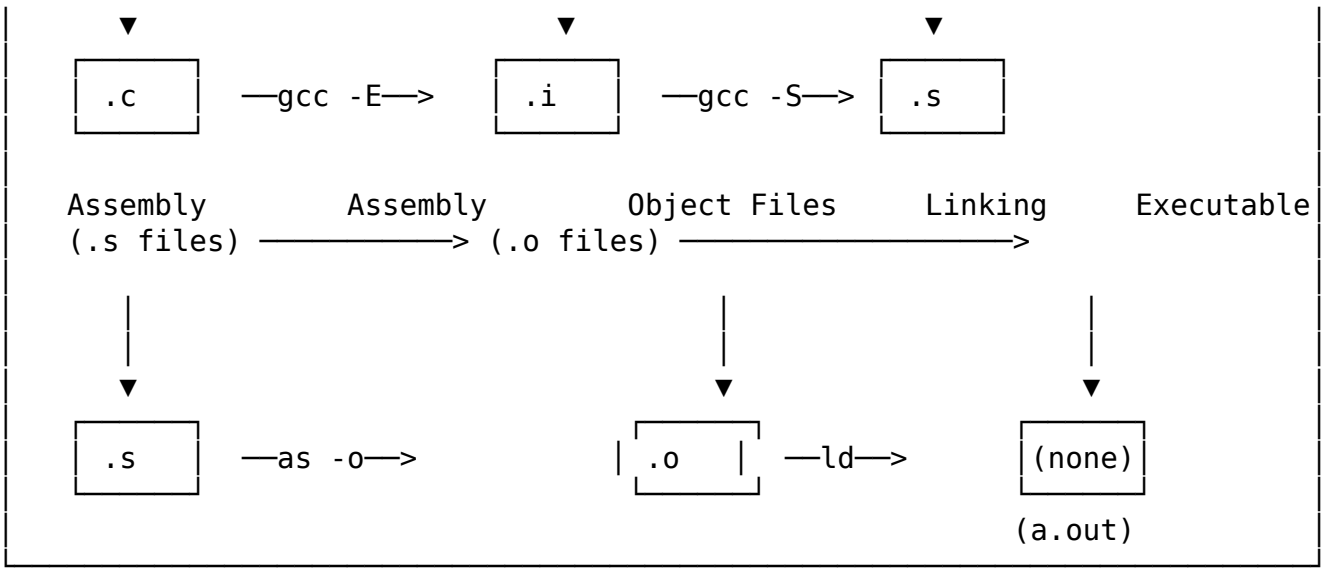
- **Debugging build failures**: Knowing which stage failed helps pinpoint the problem
- **Optimizing performance**: Optimization happens at specific stages
- **Understanding errors**: Different stages produce different kinds of errors
- **Cross-compilation**: Each stage may run on different machines
- **Build system design**: Makefiles and other build tools orchestrate these stages

This chapter provides a complete map of the compilation pipeline, showing how C source code becomes an executable program and what tools make this transformation possible.

2.2 The Complete Compilation Pipeline

The C compilation pipeline consists of four main stages, each converting one file format into another:





Each stage has a specific purpose and produces a distinct artifact:

Stage	Input	Output	Tool	Purpose
Preprocessing	`.c`	`.i`	`cpp` (via `gcc -E`)	Macro expansion, file inclusion
Compilation	`.i`	`.s`	`cc1` (via `gcc -S`)	Parse, optimize, generate assembly
Assembly	`.s`	`.o`	`as`	Encode to machine code
Linking	`.o`	executable	`ld`	Combine objects, resolve symbols

Key Insight: While we usually invoke `gcc` or `clang` to go directly from `.c` to executable, internally these drivers invoke separate programs for each stage. We can stop at any intermediate stage to inspect the artifacts.

2.2.1 Why Separate Stages Matter

The modular design of the compilation pipeline serves several important purposes:

1. **Separation of Concerns**: Each stage focuses on one transformation, making the toolchain easier to develop and maintain
2. **Optimization Opportunities**: Different optimization techniques apply at different stages. For example:
   - Preprocessor macros are expanded before compilation
   - Code optimization happens during compilation
   - Link-time optimization (LTO) can happen across translation units
3. **Debugging**: When something goes wrong, you can inspect intermediate files to understand where the problem occurred
4. **Language Interoperability**: Different frontends can produce the same intermediate representations, allowing code in different languages to be linked together

5. **Cross-compilation**: Each stage can theoretically run on different machines (e.g., compiling on a powerful build server for a target embedded system)

Let's now examine each stage in detail.

2.3 Stage 1: Preprocessing (`.c` → `.i`)

Preprocessing is the first stage of compilation. It transforms your source code through **textual substitution** before the compiler ever sees it. The preprocessor doesn't understand C syntax—it works purely on text manipulation.

2.3.1 What the Preprocessor Does

The preprocessor performs three main operations:

1. **File Inclusion**: Inserting the contents of other files (via `#include`)
2. **Macro Expansion**: Replacing macros with their definitions (via `#define`)
3. **Conditional Compilation**: Including or excluding code based on conditions (via `#if`, `#ifdef`, etc.)

Consider this example:

```
// main.c
#define MAX_SIZE 100
#define SQUARE(x) ((x) * (x))

#include <stdio.h>

#ifdef DEBUG
   #define LOG(msg) printf("DEBUG: \%s\n", msg)
#else
   #define LOG(msg) /* do nothing */
#endif

int main(void) {
   int arr[MAX_SIZE];
   LOG("Program started");
   int result = SQUARE(5);
   return 0;
}
```

After preprocessing (run `gcc -E main.c -o main.i`), we get a massive file (typically thousands of lines) where:

- `#include <stdio.h>` has been replaced with the entire contents of `stdio.h` (and all headers it includes)
- `MAX_SIZE` has been replaced with `100`
- `SQUARE(5)` has been replaced with `((5) * (5))`
- The `LOG` macro has been replaced either with `printf` or nothing, depending on whether `DEBUG` is defined

2.3.2 Preprocessing Example

Let's create a simpler example to see the transformation clearly:

```
// example.c
#define PI 3.14159
#define AREA(r) (PI * (r) * (r))

double circle_area(double radius) {
   return AREA(radius);
}
```

Running `gcc -E example.c` produces:

```
# 1 "example.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "example.c"

double circle_area(double radius) {
   return (3.14159 * (radius) * (radius));
}
```

Notice that:

- All `#define` directives are gone
- The macros have been expanded
- Preprocessor directives (lines starting with #) show the original source location for debugging

2.3.3 Translation Units

The output of preprocessing is called a **translation unit**—a single, complete source file ready for compilation. Each `.c` file becomes one translation unit, which the compiler processes independently.

This is crucial for understanding compilation:

- Each `.c` file is compiled **separately**
- The compiler doesn't see other `.c` files
- Cross-file references are resolved during **linking**

  Common Pitfall: Because each `.c` file is compiled independently, you must ensure definitions are consistent across translation units. This is why we use header files (`.h` files)—to share declarations between compilation units.

We'll explore preprocessing in depth in Chapter 2.

2.4 Stage 2: Compilation (`.i` → `.s`)

Compilation proper transforms the preprocessed source code into assembly language. This is where the heavy lifting happens: parsing, semantic analysis, optimization, and code generation.

2.4.1 What the Compiler Does

The compiler performs several sophisticated operations:

1. **Lexical Analysis**: Breaking the code into tokens (identifiers, keywords, operators)
2. **Parsing**: Building an Abstract Syntax Tree (AST) based on C grammar
3. **Semantic Analysis**: Type checking, ensuring variables are declared before use, etc.
4. **Intermediate Representation**: Converting the AST into an IR for optimization
5. **Optimization**: Applying various transformations to improve performance
6. **Code Generation**: Emitting assembly language for the target architecture

2.4.2 Compilation Example

Let's compile a simple function:

```
// add.c
int add(int a, int b) {
   return a + b;
}
```

First preprocess it:

```
gcc -E add.c -o add.i
```

Then compile to assembly:

```
gcc -S add.i -o add.s -fno-asynchronous-unwind-tables -fno-stack-protector
```

The resulting assembly (on x86-64 Linux):

```
   .intel_syntax noprefix
   .globl add
   .type add, @function
add:
   mov eax, edi
   add eax, esi
   ret
```

Even from this simple example, we can see:

- The function is marked as global (`.globl`)
- Arguments are in registers (`edi` and `esi` per the calling convention)
- The return value goes in `eax`
- The `ret` instruction returns to the caller

2.4.3 Why Assembly?

You might wonder: why generate assembly instead of going directly to machine code? Several reasons:

1. **Readability**: Assembly is human-readable, useful for debugging
2. **Portability**: The same compiler can use different assemblers for different targets
3. **Flexibility**: Developers can write hand-optimized assembly when needed
4. **Debugging**: Assembly listings with source annotations help understand what the compiler did

2.4.4 Optimization Levels

The compiler's behavior changes dramatically based on optimization levels:

```
gcc -O0 add.c -o add.o    # No optimization (default for debugging)
gcc -O1 add.c -o add.o    # Basic optimization
gcc -O2 add.c -o add.o    # Standard optimization (recommended)
gcc -O3 add.c -o add.o    # Aggressive optimization
gcc -Os add.c -o add.o    # Optimize for code size
```

With `-O2`, our `add` function becomes even simpler (the compiler might inline it entirely if it's small enough).

We'll explore compilation internals in depth in Chapter 3.

2.5 Stage 3: Assembly (`.s` → `.o`)

Assembly converts human-readable assembly language into machine code, producing an **object file**. This is a binary format, but not yet executable.

2.5.1 What the Assembler Does

The assembler:

1. **Parses Assembly**: Converts mnemonic instructions into their binary encodings
2. **Resolves Symbols**: Creates symbol table entries for labels and external references
3. **Generates Sections**: Places code and data into appropriate sections (`.text`, `.data`, etc.)
4. **Creates Relocations**: Records locations that need fixing during linking

2.5.2 Assembly Example

Let's assemble our `add.s` file:

```
as add.s -o add.o
```

The resulting `add.o` is a binary file in **ELF format** (Executable and Linkable Format). We can inspect it:

```
$ file add.o
add.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

$ ls -l add.o
-rw-r--r-- 1 user user 864 Nov 15 10:30 add.o
```

The file is only 864 bytes, containing:

- Machine code for the `add` function
- A symbol table noting that `add` is a global symbol
- Section headers marking where code and data are located
- Relocation information (none needed for this simple function)

2.5.3 Object File Contents

Object files contain several key components:

Component	Purpose
Section Headers	Describe the sections (`.text`, `.data`, etc.)
Code Sections	Machine code (`.text` for executable code)
Data Sections	Initialized data (`.data`), zero-initialized data (`.bss`)
Symbol Table	List of defined and undefined symbols
Relocation Entries	Places needing address fixups during linking
Debug Information	(optional) Debug symbols for gdb

2.5.4 Why Object Files?

Object files are **relocatable**—they don't have fixed memory addresses. This allows:

- Multiple object files to be linked together
- Code to be loaded at any memory address
- Libraries to be shared across multiple programs

The linker will ultimately resolve all symbols and assign final addresses.
We'll explore assembly and object files in depth in Chapter 4.

2.6 Stage 4: Linking (`.o` → executable)

Linking is the final stage, combining multiple object files and libraries into a single executable program. This is where separate compilation units finally come together.

2.6.1 What the Linker Does

The linker performs several critical tasks:

1. **Symbol Resolution**: Connecting references to definitions across object files
2. **Relocation**: Assigning final memory addresses and fixing references
3. **Library Linking**: Including code from standard and user libraries
4. **Executable Creation**: Setting up the executable file format with proper headers

2.6.2 Linking Example

Let's create a complete program with two files:

```c
// main.c
extern int add(int a, int b);

int main(void) {
   int result = add(5, 3);
   return result == 8 ? 0 : 1;
}
```

```c
// add.c
int add(int a, int b) {
   return a + b;
}
```

Compile and assemble separately:

```
gcc -c main.c -o main.o    # Stops after assembly
gcc -c add.c -o add.o      # Stops after assembly
```

Now link them:

```
gcc main.o add.o -o program
```

Or using the linker directly:

```
ld main.o add.o -o program \
   /usr/lib/x86_64-linux-gnu/crt1.o \
   /usr/lib/x86_64-linux-gnu/crti.o \
   /usr/lib/x86_64-linux-gnu/crtn.o \
   -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2
```

The `gcc` version is simpler because it automatically includes:

- **C runtime startup code** (`crt1.o`, `crti.o`, `crtn.o`)
- **Standard library** (`-lc` for libc)
- **Dynamic linker** path

2.6.3 Symbol Resolution

Symbol resolution is the linker's most critical job. Consider our example:

```
main.o:  UNDEFINED: add
         DEFINED:   main

add.o:   DEFINED:   add
```

The linker sees that `main.o` references `add`, finds it defined in `add.o`, and connects them. If `add` were not defined anywhere, the linker would error:

```
ld: main.o: in function 'main':
main.c:(.text+0x14): undefined reference to 'add'
```

2.6.4 Static vs. Dynamic Linking

Linking can happen at different times:

Static Linking:

- All library code is included in the executable
- Executables are larger but self-contained
- No runtime dependencies
- Command: `gcc program.o -o program -static` or `gcc program.o -o program -Wl,-Bstatic -lfoo`

Dynamic Linking (default):

- Library code is referenced but not included
- Executables are smaller
- Libraries loaded at runtime
- Allows library updates without recompiling
- Command: `gcc program.o -o program` (default)

```
$ ldd program  # Shows dynamic library dependencies
    linux-vdso.so.1
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
    /lib64/ld-linux-x86-64.so.2
```

2.6.5 Library Search Order

The linker searches for libraries in a specific order:

1. Paths specified with `-L` flags (left to right)
2. Standard system library directories (`/usr/lib`, `/lib`, etc.)

3. Paths in `LD_LIBRARY_PATH` environment variable (at runtime)

Important: The order of `-l` flags matters! Libraries are searched left-to-right, and a library is only used to resolve undefined symbols from libraries to its left.

```
# Wrong: libfoo needs symbols from libbar, but libbar is searched first
gcc program.o -o program -lfoo -lbar

# Correct: libbar is available when libfoo needs it
gcc program.o -o program -lbar -lfoo
```

We'll explore linking and relocation in depth in Chapter 6.

2.7 Toolchain Components and Roles

Now that we've seen the complete pipeline, let's identify the actual tools involved:

2.7.1 The GCC Toolchain

When you install `gcc`, you actually get a collection of programs:

```
$ which gcc cc1 as ld
/usr/bin/gcc
/usr/lib/gcc/x86_64-linux-gnu/11/cc1
/usr/bin/as
/usr/bin/ld
```

Tool	Full Name	Role	Stage
`gcc/clang`	Compiler Driver	Orchestrates the pipeline	All
`cpp`	C Preprocessor	Macro expansion, includes	Preprocessing
`cc1`	C Compiler	Parses, optimizes, generates assembly	Compilation
`as`	Assembler	Assembly to machine code	Assembly
`ld`	Linker	Combines objects, resolves symbols	Linking

> **Note**: `gcc` is technically a **compiler driver**—it doesn't do compilation itself. Instead, it invokes the appropriate tools (`cpp`, `cc1`, `as`, `ld`) in the correct order with the right arguments.

2.7.2 The Clang Toolchain

Clang follows a similar architecture:

Tool	Role
`clang`	Compiler driver (front-end)
`llvm-as`	LLVM assembler
`llc`	LLVM static compiler
`lld`	LLVM linker

2.7.3 Compiler Drivers: Why They Exist

You might wonder: if we have separate tools (`cpp`, `cc1`, `as`, `ld`), why do we typically just run `gcc`?

Compiler drivers exist to:

1. **Simplify**: Hide complexity from the user
2. **Standardize**: Provide a consistent interface across platforms
3. **Orchestrate**: Run tools in the correct order
4. **Configure**: Add necessary flags and include paths automatically

When you run:

```
gcc -O2 -Wall program.c -o program
```

The driver:

1. Determines the file type (C source)
2. Constructs the command line for `cpp`
3. Constructs the command line for `cc1` (adding `-O2` optimization)
4. Constructs the command line for `as`
5. Constructs the command line for `ld` (adding necessary libraries)
6. Runs each tool sequentially
7. Cleans up intermediate files (unless you use `-save-temps`)

2.7.4 Stopping at Intermediate Stages

You can stop the compilation process at any stage:

```
# Stop after preprocessing (keep .i file)
gcc -E program.c -o program.i

# Stop after compilation (keep .s file)
gcc -S program.c -o program.s

# Stop after assembly (keep .o file)
gcc -c program.c -o program.o

# Keep all intermediate files
gcc -save-temps program.c -o program
# Creates: program.i, program.s, program.o, program
```

This is incredibly useful for:

- **Debugging**: Seeing what the preprocessor or compiler actually did
- **Learning**: Understanding how C code transforms to assembly
- **Optimizing**: Inspecting compiler output at different optimization levels

2.7.5 Multiple Source Files

For projects with multiple source files:

```
# Compile all files separately (faster for incremental builds)
gcc -c main.c -o main.o
gcc -c util.c -o util.o
gcc -c data.c -o data.o
gcc main.o util.o data.o -o program

# Or let the driver do it (one command)
gcc main.c util.c data.c -o program
```

The second approach compiles each `.c` file to `.o`, then links them all.

2.8 A Complete Example: Tracing the Pipeline

Let's trace a complete program through all four stages:

```c
// greet.c
#include <stdio.h>

#define GREETING "Hello, World!"

int main(void) {
   printf("%s\n", GREETING);
   return 0;
}
```

2.8.1 Stage 1: Preprocessing

```
gcc -E greet.c -o greet.i
```

`greet.i` contains the preprocessed source (over 2000 lines due to `stdio.h` expansion). The key parts:

```c
# 1 "greet.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "greet.c"
# 1 "/usr/include/stdio.h" 1 3 4
... [thousands of lines from stdio.h] ...
# 2 "greet.c" 2

int main(void) {
   printf("%s\n", "Hello, World!");
   return 0;
}
```

Notice:

- `#include <stdio.h>` is gone, replaced by its contents
- `GREETING` macro has been expanded to `"Hello, World!"`

2.8.2 Stage 2: Compilation

```
gcc -S greet.i -o greet.s
```

`greet.s` contains assembly code. Simplified version:

```
   .file "greet.c"
   .intel_syntax noprefix
   .section .rodata
.LC0:
   .string "Hello, World!"
   .text
   .globl main
   .type main, @function
main:
   push rbp
   mov rbp, rsp
   sub rsp, 16
   lea rdi, [rip + .LC0]
   call printf
   mov eax, 0
   leave
   ret
   .size main, .-main
```

Key observations:

- The string literal is in the `.rodata` section (read-only data)
- `printf` is called with the string address
- Return value 0 is placed in `eax`

2.8.3 Stage 3: Assembly

```
as greet.s -o greet.o
```

`greet.o` is now a binary object file:

```
$ file greet.o
greet.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

$ size greet.o
   text    data     bss     dec     hex filename
    135       8       0     143      8f greet.o
```

The object file contains:

- 135 bytes of code (`.text`)
- 8 bytes of data (the string in `.rodata`)
- A symbol table marking `main` as defined and `printf` as undefined
- Relocation entries for the string reference and `printf` call

2.8.4 Stage 4: Linking

```
gcc greet.o -o greet
# Or explicitly:
ld greet.o -o greet \
   /usr/lib/x86_64-linux-gnu/crt1.o \
   /usr/lib/x86_64-linux-gnu/crti.o \
   /usr/lib/x86_64-linux-gnu/crtn.o \
   -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2
```

Now `greet` is an executable:

```
$ file greet
greet: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, in

$ ls -l greet
-rwxr-xr-x 1 user user 16856 Nov 15 10:45 greet

$ ./greet
Hello, World!
```

2.9 Key Takeaways

1. **The C compilation pipeline has four stages**: Preprocessing, Compilation, Assembly, and Linking. Each stage transforms one file format into another.
2. **Preprocessing (`.c` → `.i`)**: Textual substitution through macro expansion, file inclusion, and conditional compilation. Creates translation units.
3. **Compilation (`.i` → `.s`)**: Parses code, builds an AST, generates intermediate representation, applies optimizations, and emits assembly language.
4. **Assembly (`.s` → `.o`)**: Converts assembly to machine code, creating relocatable object files with symbol tables and relocations.
5. **Linking (`.o` → executable)**: Combines object files, resolves symbols, applies relocations, and creates the final executable with proper runtime setup.
6. **Toolchain components**: `gcc`/`clang` are compiler drivers that orchestrate `cpp`, `cc1`, `as`, and `ld`. Understanding each tool helps debug build issues.
7. **Separate compilation**: Each `.c` file becomes one translation unit, compiled independently. Cross-file references are resolved during linking.
8. **You can stop at any stage**: Use `-E` (preprocessor), `-S` (compilation), `-c` (assembly), or `-save-temps` (keep everything) to inspect intermediate artifacts.
9. **Optimization levels matter**: `-O0` (fast compilation, good for debugging), `-O2` (recommended for production), `-O3` (maximum optimization), `-Os` (optimize for size).
10. **Understanding the pipeline helps**: Debug build failures, optimize performance, interpret compiler errors, and design effective build systems.

2.10 Looking Ahead

Now that we've seen the complete compilation pipeline, we'll examine each stage in detail:

- **Chapter 2**: Preprocessing—how macros work, include file handling, conditional compilation
- **Chapter 3**: Compilation internals—lexers, parsers, ASTs, IR, optimization
- **Chapter 4**: Assembly and object files—assembler syntax, object file structure
- **Chapter 5**: ELF format—headers, sections, segments, symbols, relocations
- **Chapter 6**: Linking and relocation—symbol resolution, static vs. dynamic linking

Chapter 3

Preprocessing

3.1 Introduction

Before a C compiler sees your source code, it passes through the **preprocessor**—a powerful text transformation engine that manipulates your code at the character level. The preprocessor doesn't understand C syntax; it simply performs textual substitutions, file inclusions, and conditional deletions.

Understanding preprocessing is essential because:

- It's where macros, includes, and conditionals are handled
- Errors in preprocessing can produce baffling compiler errors
- The preprocessor determines what the compiler actually sees
- Many build configurations rely on preprocessor conditionals

In this chapter, we'll explore every aspect of the C preprocessor, from simple macros to complex conditional compilation, with hands-on examples you can try yourself.

3.2 What is the Preprocessor?

The C preprocessor (often abbreviated as CPP) is a separate program that processes your source code before the compiler proper. On most systems, it's invoked automatically by the compiler driver (like `gcc`), but you can also run it standalone:

```
# Run preprocessor only, output to stdout
gcc -E source.c

# Run preprocessor only, output to file
gcc -E source.c -o source.i
```

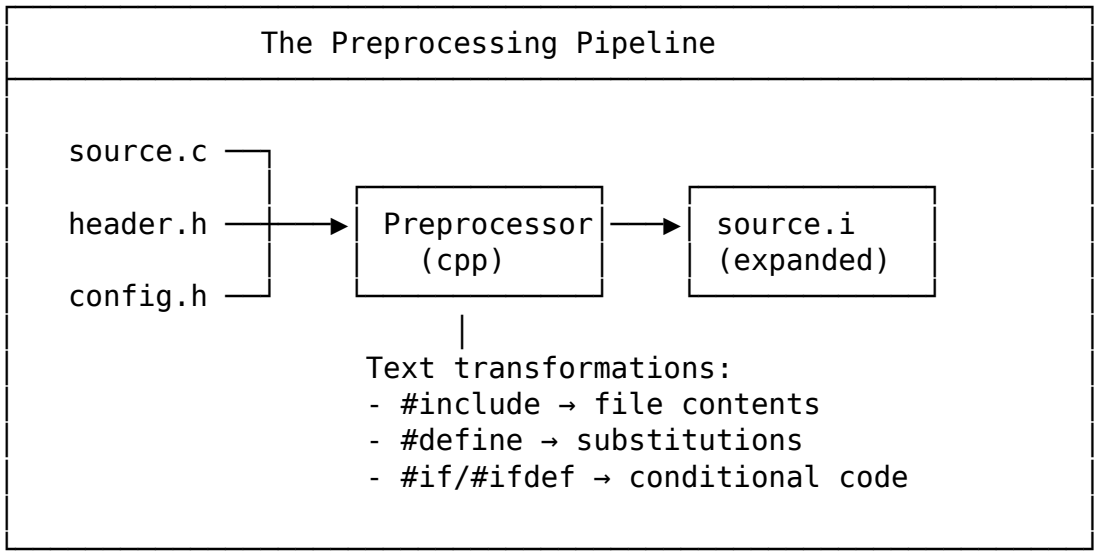
The preprocessor operates on **translation units**—essentially, what the compiler processes as a single entity. The output of the preprocessor is a single, expanded text stream that the compiler then parses.

3.2.1 Preprocessing as Text Transformation

The preprocessor performs three main operations:

1. **Macro expansion**: Replace defined symbols with their definitions
2. **File inclusion**: Insert the contents of other files
3. **Conditional compilation**: Include or exclude code based on conditions

All of these are pure text operations. The preprocessor doesn't know about C syntax, types, or scope. It just manipulates text.



3.3 Macro Expansion

3.3.1 Object-Like Macros

The simplest macros are object-like—they replace a name with a token sequence:

```
#define PI 3.14159
#define MAX_SIZE 1024
#define DEBUG 1

int main(void) {
   double area = PI * radius * radius;
   int buffer[MAX_SIZE];
   if (DEBUG) {
      printf("Debug mode enabled\n");
   }
   return 0;
}
```

After preprocessing, this becomes:

```
int main(void) {
   double area = 3.14159 * radius * radius;
   int buffer[1024];
   if (1) {
      printf("Debug mode enabled\n");
```

```
    }
    return 0;
}
```

Tip

Use UPPERCASE names for macros to distinguish them from variables. This is a convention, not a requirement, but it helps readers identify macros at a glance.

3.3.2 Function-Like Macros

Macros can take arguments, making them look like functions:

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define SQUARE(x) ((x) * (x))
#define ABS(x) ((x) < 0 ? -(x) : (x))

int main(void) {
    int m = MAX(3, 5); // Expands to: ((3) > (5) ? (3) : (5))
    int sq = SQUARE(4); // Expands to: ((4) * (4))
    int av = ABS(-10); // Expands to: ((-10) < 0 ? -(-10) : (-10))
    return 0;
}
```

Warning

Critical Warning: Always parenthesize macro arguments! Without parentheses, operator precedence can cause unexpected behavior:

```
#define BAD_SQUARE(x) x * x
int result = BAD_SQUARE(1 + 2); // Expands to: 1 + 2 * 1 + 2 = 5, not 9!
```

3.3.3 Macro Pitfall: Multiple Evaluation

Function-like macros can evaluate arguments multiple times:

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int x = 5;
int m = MAX(x++, 3); // Problem: x might be incremented once or twice!
```

After expansion:

```
int m = ((x++) > (3) ? (x++) : (3));
```

If `x` is 5, then `x++` (which returns 5) is compared to 3. Since 5 > 3 is true, the result is `x++` again—but now `x` is 6, so this returns 6. `x` ends up as 7, and `m` is 6.

Solution: Use inline functions when possible, or be very careful with macro arguments that have side effects:

```
// Prefer inline functions for type safety and single evaluation
static inline int max(int a, int b) {
   return a > b ? a : b;
}
```

3.3.4 Multi-Line Macros

For longer macros, use backslash continuation:

```
#define SWAP(a, b, type) do { \
   type temp = a; \
   a = b; \
   b = temp; \
} while (0)

int main(void) {
   int x = 1, y = 2;
   SWAP(x, y, int); // x = 2, y = 1
   return 0;
}
```

Tip

Why the do { ... } while (0)? This idiom allows the macro to be used like a statement with a semicolon, without breaking if-else chains:

```
if (condition)
   SWAP(a, b, int); // Works correctly
else
   do_something();
```

3.3.5 Stringification (#)

The # operator converts a macro argument to a string:

```
#define STRINGIFY(x) #x
#define TOSTRING(x) STRINGIFY(x)

#define VALUE 42

const char* s1 = STRINGIFY(VALUE); // s1 = "VALUE"
const char* s2 = TOSTRING(VALUE); // s2 = "42"
```

The difference: `STRINGIFY` stringifies the argument literally, while `TOSTRING` first expands the argument, then stringifies it.

3.3.6 Token Pasting (##)

The ## operator concatenates tokens:

```c
#define CONCAT(a, b) a ## b
#define MAKE_VAR(name, num) int name ## _ ## num

int main(void) {
   int CONCAT(var, 1) = 10; // Creates variable var1
   MAKE_VAR(item, 5) = 20; // Creates variable item_5
   return 0;
}
```

This is commonly used for generating names programmatically:

```c
#define DEFINE_HANDLER(num) void handler_##num(void) { \
   printf("Handler␣%d␣called\n", num); \
}

DEFINE_HANDLER(1) // Creates handler_1
DEFINE_HANDLER(2) // Creates handler_2
DEFINE_HANDLER(3) // Creates handler_3
```

3.3.7 Variadic Macros

Macros can accept variable numbers of arguments using `...` and `__VA_ARGS__`:

```c
#define DEBUG_PRINT(fmt, ...) \
   fprintf(stderr, "[DEBUG]␣" fmt "\n", __VA_ARGS__)

#define LOG(level, msg, ...) \
   printf("[%s]␣" msg "\n", level, __VA_ARGS__)

int main(void) {
   DEBUG_PRINT("Value:␣%d,␣Status:␣%s", 42, "OK");
   LOG("INFO", "User␣%s␣logged␣in", "alice");
   return 0;
}
```

For C99 compatibility with zero variadic arguments, use `#__VA_ARGS__` (GCC extension, now standard in C++20):

```c
#define DEBUG_PRINT(fmt, ...) \
   fprintf(stderr, "[DEBUG]␣" fmt "\n", ##__VA_ARGS__)

DEBUG_PRINT("Simple␣message"); // Works even with no extra args
```

3.4 File Inclusion

3.4.1 #include Basics

The `#include` directive inserts the contents of another file:

```c
#include <stdio.h> // System header (angle brackets)
#include "myheader.h" // Local header (double quotes)
```

The difference between `<...>` and `"..."` is the search path:

Syntax	Search Order
`<header.h>`	System include paths only
`"header.h"`	Current directory first, then system include paths

3.4.2 Include Search Paths

The preprocessor searches for headers in specific directories:

```
# View default include paths
gcc -E -x c - -v < /dev/null 2>&1 | grep "^␣"

# Add custom include paths
gcc -I/path/to/includes -I../another/path source.c
```

Common system include paths on Linux:

- `/usr/include`
- `/usr/local/include`
- `/usr/lib/gcc/x86_64-linux-gnu/11/include` (compiler-specific)

3.4.3 The Include Graph

With nested includes, you can build complex dependency graphs:

```
source.c
├── <stdio.h>
│   ├── <bits/libc-header-start.h>
│   ├── <features.h>
│   │   ├── <stdc-predef.h>
│   │   └── <sys/cdefs.h>
│   └── <bits/types.h>
├── "config.h"
│   └── <stdint.h>
└── "utils.h"
    └── <stdlib.h>
```

To see the include graph:

```
# Show included files
gcc -E source.c -H

# Output example:
# . /usr/include/stdio.h
# .. /usr/include/bits/libc-header-start.h
# .. /usr/include/features.h
```

3.4.4 Include Guards and #pragma once

Without protection, a header included twice causes errors:

```
// header.h
struct Point { int x, y; };

// source.c
#include "header.h"
#include "header.h" // Error: redefinition of 'struct Point'!
```

Solution 1: Include Guards (Traditional)

```
#ifndef HEADER_H
#define HEADER_H

struct Point { int x, y; };

#endif /* HEADER_H */
```

The first inclusion defines `HEADER_H`. Subsequent inclusions see the guard and skip the content.

Solution 2: #pragma once (Modern)

```
#pragma once

struct Point { int x, y; };
```

`#pragma once` tells the preprocessor to include this file at most once per compilation unit. It's simpler and often faster, but technically non-standard (though supported by all major compilers).

Recommendation

Use `#pragma once` for new code. It's cleaner and less error-prone. Use include guards if you need strict C standard compliance.

3.5 Conditional Compilation

Conditional compilation lets you include or exclude code based on conditions evaluated during preprocessing.

3.5.1 #if, #elif, #else, #endif

```
#define VERSION 3

#if VERSION == 1
   #define FEATURE_NAME "Basic"
#elif VERSION == 2
   #define FEATURE_NAME "Standard"
#elif VERSION == 3
   #define FEATURE_NAME "Premium"
#else
```

```
   #define FEATURE_NAME "Unknown"
#endif
```

3.5.2 #ifdef and #ifndef

Test whether a macro is defined:

```
#ifdef DEBUG
   printf("Debug mode enabled\n");
#endif

#ifndef BUFFER_SIZE
   #define BUFFER_SIZE 1024 // Default value
#endif
```

These are equivalent to:

```
#if defined(DEBUG)
#if !defined(BUFFER_SIZE)
```

3.5.3 defined() Operator

The `defined()` operator checks if a macro exists:

```
#if defined(DEBUG) && defined(VERBOSE)
   #define LOG_LEVEL 3
#elif defined(DEBUG)
   #define LOG_LEVEL 2
#else
   #define LOG_LEVEL 1
#endif
```

3.5.4 Common Use Cases

Platform Detection:

```
#ifdef _WIN32
   #define PATH_SEPARATOR '\\'
   #include <windows.h>
#elif defined(__linux__)
   #define PATH_SEPARATOR '/'
   #include <unistd.h>
#elif defined(__APPLE__)
   #define PATH_SEPARATOR '/'
   #include <mach-o/dyld.h>
#endif
```

Debug vs. Release:

```
#ifdef NDEBUG
   #define ASSERT(cond) ((void)0)
#else
   #define ASSERT(cond) \
      do { \
```

```
         if (!(cond)) { \
            fprintf(stderr, "Assertion␣failed:␣%s,␣%s:%d\n", \
                  #cond, __FILE__, __LINE__); \
            abort(); \
         } \
      } while (0)
#endif
```

Feature Flags:

```
// Compile with: gcc -DUSE_OPENCL source.c
#ifdef USE_OPENCL
   void compute(void) { /* OpenCL implementation */ }
#elif defined(USE_CUDA)
   void compute(void) { /* CUDA implementation */ }
#else
   void compute(void) { /* CPU fallback */ }
#endif
```

Header Inclusion Control:

```
// Disable assert.h assertions
#define NDEBUG
#include <assert.h>
```

3.6 Predefined Macros

The C standard and compilers provide predefined macros:

3.6.1 Standard Predefined Macros

Macro	Description	Example Value
`__FILE__`	Current source file name	`"source.c"`
`__LINE__`	Current line number	`42`
`__func__`	Current function name (C99)	`"main"`
`__DATE__`	Compilation date	`"Mar 11 2026"`
`__TIME__`	Compilation time	`"16:30:00"`
`__STDC__`	1 if compiler conforms to C standard	`1`
`__STDC_VERSION__`	C standard version (as long)	`201710L` (C17)

3.6.2 Compiler-Specific Macros

GCC:

```
#ifdef __GNUC__
   #define GCC_VERSION (__GNUC__ * 10000 + __GNUC_MINOR__ * 100 +
       __GNUC_PATCHLEVEL__)
   #if GCC_VERSION >= 90100
      // GCC 9.1+ specific code
   #endif
#endif
```

Clang:

```
#ifdef __clang__
   #define CLANG_VERSION (__clang_major__ * 10000 + __clang_minor__ * 100 +
       __clang_patchlevel__)
#endif
```

MSVC:

```
#ifdef _MSC_VER
   #if _MSC_VER >= 1920
      // Visual Studio 2019+
   #endif
#endif
```

3.6.3 Platform Detection Macros

```
// Architecture
#if defined(__x86_64__) || defined(_M_X64)
   #define ARCH_X64
#elif defined(__i386__) || defined(_M_IX86)
   #define ARCH_X86
#elif defined(__arm__) || defined(_M_ARM)
   #define ARCH_ARM
#elif defined(__aarch64__)
   #define ARCH_ARM64
#endif

// Operating System
#if defined(_WIN32) || defined(_WIN64)
   #define OS_WINDOWS
#elif defined(__linux__)
   #define OS_LINUX
#elif defined(__APPLE__)
   #define OS_MACOS
#elif defined(__FreeBSD__)
   #define OS_FREEBSD
#endif
```

3.6.4 Practical Example: Debug Logging

```
#define LOG_DEBUG(fmt, ...) \
   fprintf(stderr, "[%s:%d] " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__)

#define LOG_FUNC_ENTRY() \
   fprintf(stderr, "Entering %s() at %s:%d\n", __func__, __FILE__, __LINE__)

void process_data(int value) {
   LOG_FUNC_ENTRY();
   LOG_DEBUG("Processing value: %d", value);
   // ... processing code ...
}
```

```
int main(void) {
   process_data(42);
   return 0;
}
```

Output:

```
Entering process_data() at source.c:10
[source.c:11] Processing value: 42
```

3.7 #define vs. enum vs. const

When should you use macros versus other constants?

3.7.1 Macros

```
#define MAX_ITEMS 100
#define VERSION "1.0.0"
```

Pros:

- Works in preprocessor conditionals
- No memory allocation (text substitution)
- Can be undefined and redefined

Cons:

- No type checking
- Not visible in debugger
- Can cause unexpected substitutions

3.7.2 enum

```
enum {
   MAX_ITEMS = 100,
   BUFFER_SIZE = 1024
};

// Typed enums (C23 or earlier with compiler support)
typedef enum Status {
   STATUS_OK = 0,
   STATUS_ERROR = 1,
   STATUS_PENDING = 2
} Status;
```

Pros:

- Type-safe (with typed enums)

- Visible in debugger
- Scoped (with `enum class` in C++)

Cons:

- Integer values only
- Cannot be used in preprocessor conditionals

3.7.3 const

```
const int max_items = 100;
const char* const version = "1.0.0";
```

Pros:

- Type-safe
- Scoped properly
- Visible in debugger
- Can take address

Cons:

- Cannot be used in preprocessor conditionals
- May occupy memory (though compilers often optimize)
- Cannot be used for array sizes in C89 (VLA in C99)

3.7.4 Recommendation

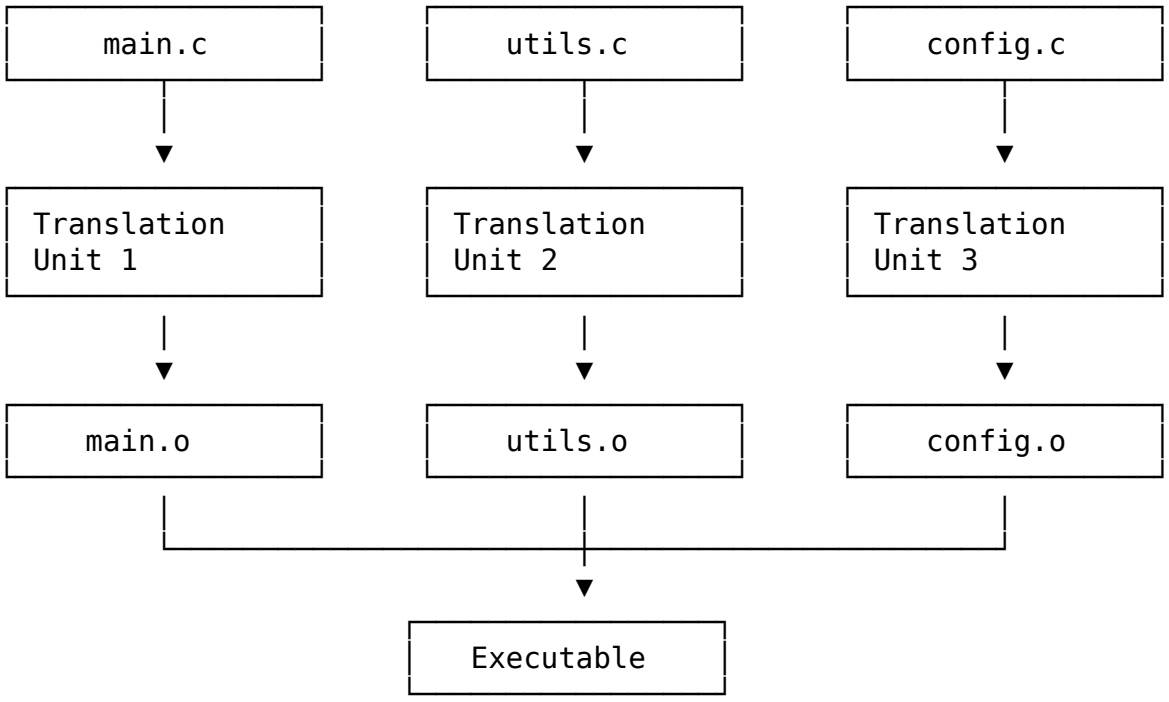
```
// For preprocessor conditionals
#define FEATURE_ENABLED 1
#define PLATFORM_LINUX 1

// For integer constants
enum { MAX_ITEMS = 100, BUFFER_SIZE = 1024 };

// For typed constants
static const char* const APP_NAME = "MyApp";
static const double PI = 3.141592653589793;
```

3.8 Translation Units

A **translation unit** is the result of preprocessing a single source file with all its included headers. This is what the compiler proper processes.

3.8.1 Translation Unit Boundaries

Each `.c` file is a separate translation unit:



3.8.2 Static Variables and Translation Units

Variables and functions with `static` linkage are private to their translation unit:

```
// utils.c
static int internal_counter = 0; // Only visible in utils.c

static void internal_helper(void) { // Only visible in utils.c
   internal_counter++;
}

void public_function(void) {
   internal_helper();
}

// main.c
extern int internal_counter; // Error! Not accessible
```

3.8.3 One Definition Rule (ODR)

Each translation unit can have at most one definition of each external symbol. Violations cause linker errors:

```
// header.h
int global_var = 42; // WRONG! Multiple definitions if included in multiple .c
    files
```

```
// Correct approach:
// header.h
extern int global_var; // Declaration only

// source.c
int global_var = 42; // Definition in one place
```

3.9 Common Preprocessor Idioms

3.9.1 Include Guard with #pragma once

```
#ifndef MYHEADER_H
#define MYHEADER_H
#pragma once

// Header contents

#endif /* MYHEADER_H */
```

This provides both compatibility and optimization.

3.9.2 Compile-Time Assertions

```
// C11 _Static_assert
_Static_assert(sizeof(int) == 4, "int must be 4 bytes");
static_assert(sizeof(void*) == 8, "64-bit platform required");

// Pre-C11 hack
#define STATIC_ASSERT(cond, msg) \
   typedef char static_assert_##msg[(cond) ? 1 : -1]

STATIC_ASSERT(sizeof(int) == 4, int_must_be_4_bytes);
```

3.9.3 X-Macro Pattern

Generate repetitive code from a data table:

```
// Define the list
#define ITEM_LIST \
   X(ITEM_A, 1) \
   X(ITEM_B, 2) \
   X(ITEM_C, 3)

// Generate enum
typedef enum {
   #define X(name, value) name = value,
   ITEM_LIST
   #undef X
} Items;
```

```c
// Generate string array
const char* item_names[] = {
   #define X(name, value) #name,
   ITEM_LIST
   #undef X
};

// Generate handler functions
#define X(name, value) void handle_##name(void);
ITEM_LIST
#undef X

// Expands to:
// typedef enum { ITEM_A = 1, ITEM_B = 2, ITEM_C = 3 } Items;
// const char* item_names[] = { "ITEM_A", "ITEM_B", "ITEM_C" };
// void handle_ITEM_A(void);
// void handle_ITEM_B(void);
// void handle_ITEM_C(void);
```

3.9.4 Counting Arguments

```c
#define VA_NARGS_IMPL(_1, _2, _3, _4, N, ...) N
#define VA_NARGS(...) VA_NARGS_IMPL(__VA_ARGS__, 4, 3, 2, 1)

VA_NARGS(a) // 1
VA_NARGS(a, b) // 2
VA_NARGS(a, b, c) // 3
```

3.10 Key Takeaways

1. **Preprocessing is text-only**: The preprocessor doesn't understand C syntax; it performs pure text substitutions.
2. **Parenthesize macro arguments**: Without parentheses, operator precedence can cause unexpected results.
3. **Beware of multiple evaluation**: Function-like macros can evaluate arguments multiple times, causing side effects.
4. **Use `#pragma once` for headers**: It's simpler and less error-prone than traditional include guards.
5. **Conditional compilation is powerful**: Use it for platform detection, debug builds, and feature flags.
6. **Predefined macros are useful**: `__FILE__`, `__LINE__`, and `__func__` enable powerful debugging macros.
7. **Understand translation units**: Each `.c` file is a separate translation unit, and static symbols are private to their unit.

8. **Prefer alternatives when appropriate**: Use `enum` or `const` instead of macros for values that don't need preprocessor conditions.

3.11 Looking Ahead

Now that we understand how preprocessing transforms source code into translation units, we're ready to explore what happens next: compilation. In Chapter 3, we'll examine how the compiler parses C code, builds intermediate representations, performs optimizations, and generates assembly code.

3.12 Further Reading

- "The C Programming Language" (K&R) - Chapter 4
- C17 Standard (ISO/IEC 9899:2018) - Section 6.10 (Preprocessing directives)
- GCC Preprocessor Documentation: https://gcc.gnu.org/onlinedocs/cpp/
- "Modern C" by Jens Gustedt - Chapter 5 (Preprocessing)

Chapter 4

Compilation Internals

4.1 Introduction

After preprocessing, the compiler's real work begins. The compilation phase transforms your C source code—a human-readable text format—into assembly language that the machine can understand. This is the most complex stage of the build pipeline, involving lexical analysis, parsing, semantic analysis, optimization, and code generation.

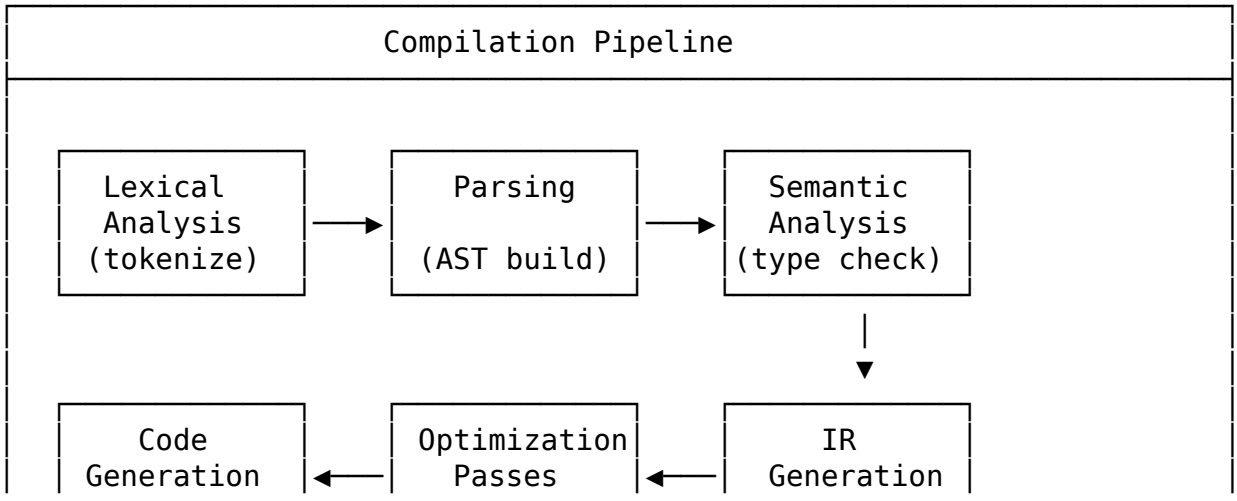
Understanding compilation internals helps you:

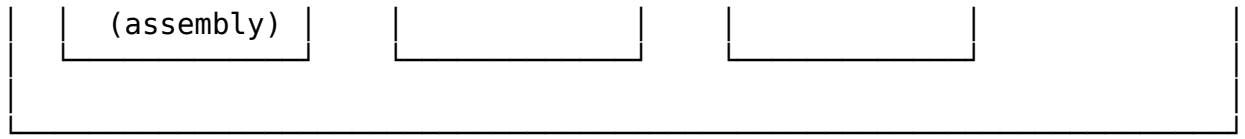
- Write code that compilers can optimize effectively
- Debug compiler errors more efficiently
- Understand why seemingly equivalent code performs differently
- Appreciate the sophisticated machinery behind modern compilers

In this chapter, we'll trace the complete journey from preprocessed source to assembly output.

4.2 The Compilation Pipeline Overview

The compiler processes code through several distinct phases:





Each phase builds on the previous one, creating increasingly refined representations of your code.

4.3 Lexical Analysis (Tokenization)

4.3.1 What is Lexical Analysis?

Lexical analysis (or scanning) converts the raw character stream of your source code into a sequence of **tokens**—the smallest meaningful units in the language. The lexer doesn't understand syntax; it just recognizes patterns.

Example input:

```
int x = 42 + y;
```

Becomes tokens:

```
[KEYWORD: int] [IDENTIFIER: x] [OPERATOR: =] [INTEGER: 42] [OPERATOR: +] [IDENTIFI
```

4.3.2 Token Categories

C tokens fall into several categories:

Category	Examples
Keywords	`int`, `if`, `else`, `while`, `return`
Identifiers	`main`, `printf`, `count`, `x`
Constants	`42`, `3.14`, `'a'`, `"hello"`
Operators	`+`, `-`, `*`, `/`, `=`, `==`, `&&`
Punctuation	`;`, `,`, `(`, `)`, `{`, `}`
Preprocessor	`#include`, `#define` (handled earlier)

4.3.3 How Lexers Work

Lexers use **regular expressions** and **finite automata** to recognize tokens:

```
Pattern for integer literal: [0-9]+
Pattern for identifier:      [a-zA-Z_][a-zA-Z0-9_]*
Pattern for keyword 'int':   int (but not followed by identifier chars)
```

The Longest Match Rule: When multiple patterns match, the lexer chooses the longest match:

- `integer` → IDENTIFIER (not `int` + `eger`)
- `while1` → IDENTIFIER (not `while` + `1`)

The Priority Rule: When patterns have the same length, keywords take priority over identifiers:

- `while` → KEYWORD (not IDENTIFIER)

4.4 Parsing and Syntax Analysis

4.4.1 What is Parsing?

Parsing (syntax analysis) takes the token stream and builds a **parse tree** or **Abstract Syntax Tree (AST)** according to the grammar rules of C. The parser checks whether the token sequence forms valid C syntax.

4.4.2 Context-Free Grammars

C's syntax is defined by a **context-free grammar (CFG)**. A grammar has:

- **Terminals**: Tokens from the lexer
- **Non-terminals**: Syntactic categories (expression, statement, declaration)
- **Production rules**: How non-terminals expand

Simplified grammar for expressions:

```
expr        → expr '+' term | expr '-' term | term
term        → term '*' factor | term '/' factor | factor
factor      → NUMBER | IDENT | '(' expr ')'
```

This grammar correctly handles operator precedence:

- `1 + 2 * 3` parses as `1 + (2 * 3)`, not `(1 + 2) * 3`

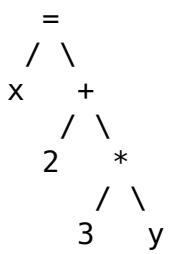
4.4.3 Parse Trees vs. Abstract Syntax Trees

Parse Tree (Concrete Syntax Tree): Contains every grammatical element, including punctuation.

Abstract Syntax Tree (AST): Eliminates syntactic sugar, focuses on structure.

For expression `x = 2 + 3 * y`:

AST (cleaner):



The AST is what the compiler uses for further processing.

4.4.4 AST Node Types

Different constructs produce different AST node types. For example, a function definition:

```
int add(int a, int b) {
   return a + b;
}
```

Produces an AST node:

```
// FunctionDecl {
//   return_type: int,
//   name: "add",
//   params: [Param{name:"a", type:int}, Param{name:"b", type:int}],
//   body: CompoundStmt {
//     body: [ ReturnStmt { expr: BinaryOp{op:'+', left:'a', right:'b'} } ]
//   }
// }
```

4.5 Semantic Analysis

4.5.1 What is Semantic Analysis?

Syntax analysis ensures the code is *syntactically* valid. Semantic analysis checks whether the code is *meaningfully* valid:

- **Type checking**: Are operations applied to compatible types?
- **Symbol resolution**: Do referenced variables/functions exist?
- **Scope management**: Are variables used in the correct scope?
- **Const correctness**: Are const variables being modified?

4.5.2 The Symbol Table

The compiler maintains a **symbol table** mapping names to their declarations. For example:

```
int global_var = 10;

int add(int a, int b) {
   int result = a + b;
   return result;
}
```

Symbol table (simplified):

```
Global Scope:
  global_var → { type: int, storage: extern, location: .data }
  add        → { type: int(int,int), storage: extern, params: [a, b] }

add function scope:
  a         → { type: int, storage: param, location: rdi }
  b         → { type: int, storage: param, location: rsi }
  result    → { type: int, storage: auto, location: stack:-4 }
```

4.6 Intermediate Representation (IR)

4.6.1 Why IR?

Directly translating AST to assembly is impractical because:

- The AST is too high-level for optimization
- Different target architectures have different instruction sets
- Optimization is easier on a simpler, more regular form

The solution is an **Intermediate Representation (IR)** that:

- Is architecture-independent (mostly)
- Is simpler than the AST
- Supports various optimizations
- Can be easily translated to assembly

4.6.2 Three-Address Code (TAC)

A simple IR form is **three-address code**, where each instruction has at most three operands:

```
// Source
int result = (a + b) * (c - d);

// Three-address code
t1 = a + b
t2 = c - d
t3 = t1 * t2
result = t3
```

Each instruction is simple: `dest = op1 operator op2` or `dest = operator op1`.

4.6.3 SSA Form (Static Single Assignment)

Static Single Assignment (SSA) is a modern IR form where each variable is assigned exactly once:

```
// Original code
int x = 1;
x = x + 2;
x = x * 3;

// SSA form
x1 = 1
x2 = x1 + 2
x3 = x2 * 3
```

SSA makes many optimizations simpler because:

- Each use refers to exactly one definition

- Def-use chains are explicit
- Data flow analysis is easier

4.6.4 LLVM IR Example

LLVM is a popular compiler infrastructure with its own well-defined IR:

```
// Source
int add(int a, int b) {
   return a + b;
}

// LLVM IR
define i32 @add(i32 %a, i32 %b) {
entry:
   %result = add i32 %a, %b
   ret i32 %result
}
```

4.7 Optimization Passes

4.7.1 Why Optimize?

C compilers perform optimizations to:

- Generate faster code
- Reduce code size
- Use registers efficiently
- Eliminate redundant computations

Modern compilers have hundreds of optimization passes.

4.7.2 Optimization Levels

Level	Description	Key Optimizations
-O0	No optimization (default)	None
-O1	Basic optimizations	Dead code elimination, basic block reorderin
-O2	Standard optimizations	Inlining, loop optimizations, common subexpr
-O3	Aggressive optimizations	Vectorization, function inlining (aggressive),
-Os	Optimize for size	Disables size-increasing optimizations
-Ofast	Maximum performance (non-standard)	-O3 + fast math + non-IEEE compliance

4.7.3 Common Optimizations

Constant Folding: Evaluate constant expressions at compile time:

```
// Source
int x = 2 + 3 * 4;

// Optimized (evaluated at compile time)
int x = 14;
```

Dead Code Elimination: Remove code that has no effect:

```
// Source
int x = 10;
x = 20; // First assignment is dead
return x;

// Optimized
int x = 20;
return x;
```

Loop Invariant Code Motion: Move computations out of loops:

```
// Source
for (int i = 0; i < n; i++) {
   x = a * b + i; // a * b is invariant
}

// Optimized
int temp = a * b;
for (int i = 0; i < n; i++) {
   x = temp + i;
}
```

Function Inlining: Replace function call with the function body:

```
// Source
static int square(int x) { return x * x; }
int result = square(5);

// Optimized
int result = 5 * 5;
```

4.8 Code Generation

4.8.1 Instruction Selection

The code generator selects machine instructions to implement each IR operation:

```
IR: t3 = t1 + t2

x86-64:  add eax, ecx      ; Add ecx to eax
ARM:     add r0, r1, r2    ; r0 = r1 + r2
RISC-V:  add a0, a1, a2    ; a0 = a1 + a2
```

4.8.2 Register Allocation

The **register allocator** maps IR temporaries to physical registers. Since registers are limited, values may need to be **spilled** to memory.

```
IR temporaries: t1, t2, t3, t4, t5, t6
Available registers: eax, ecx, edx (3 registers)

Allocation:
  t1 → eax
  t2 → ecx
  t3 → edx
  t4 → [stack - 4]   (spilled)
  t5 → eax (t1 dead, can reuse)
  t6 → ecx (t2 dead, can reuse)
```

4.8.3 Complete Code Generation Example

Source:

```
int square(int x) {
   return x * x;
}
```

IR:

```
define i32 @square(i32 %x) {
  %result = mul i32 %x, %x
  ret i32 %result
}
```

x86-64 Assembly:

```
square:
   mov eax, edi ; Move argument to eax (result register)
   imul eax, eax ; eax = eax * eax
   ret ; Return (eax contains result)
```

4.9 Compiler Flags for Optimization

4.9.1 -O0 (No Optimization)

- Fastest compilation
- Best debugging experience
- Variables stay in memory as declared
- No optimization surprises

Use for: Development, debugging

4.9.2 -O2 (Standard Optimization)

- Slower compilation
- Most beneficial optimizations
- Generally safe
- Industry standard for release builds

Optimizations enabled: all from -O1, plus inlining, loop optimizations, common subexpression elimination, instruction scheduling

4.9.3 -O3 (Aggressive Optimization)

- Slowest compilation
- May increase code size
- May not be faster than -O2
- Some risky optimizations

4.10 Viewing Compiler Output

4.10.1 View Intermediate Representations

```
# GCC: View GIMPLE
gcc -fdump-tree-gimple source.c

# GCC: View all passes
gcc -fdump-tree-all source.c

# Clang/LLVM: View LLVM IR
clang -S -emit-llvm source.c -o source.ll

# Clang/LLVM: View IR with optimization
clang -S -emit-llvm -O2 source.c -o source.ll
```

4.10.2 View Assembly Output

```
# Generate assembly file
gcc -S source.c -o source.s

# With comments and C source intermixed
gcc -S -fverbose-asm source.c -o source.s

# With source line annotations
gcc -g -S source.c -o source.s
```

4.11 Key Takeaways

1. **Compilation is multi-stage**: Lexical analysis → parsing → semantic analysis → IR generation → optimization → code generation.
2. **AST captures structure**: The Abstract Syntax Tree represents your code's syntactic structure without syntactic sugar.
3. **IR enables optimization**: Intermediate representations like SSA make optimizations easier and architecture-independent.
4. **SSA is crucial**: Static Single Assignment form, where each variable is assigned once, simplifies many analyses.
5. **Optimization levels trade off compile time vs. performance**: -O0 for debugging, -O2 for production, -O3 for maximum performance.
6. **Register allocation is critical**: Limited registers must be carefully allocated; spilling to memory hurts performance.
7. **Modern compilers are sophisticated**: Hundreds of optimization passes work together to generate efficient code.
8. **Understanding compilation helps write better code**: Knowing how compilers think helps you write code they can optimize effectively.

4.12 Looking Ahead

Now that we've seen how the compiler generates assembly code, we'll examine that assembly output in detail. In Chapter 4, we'll explore assembly language, object file structure, and how assembly is translated into machine code.

Chapter 5
Assembly and Object Files

5.1 Introduction

After the compiler generates assembly code, the **assembler** translates that assembly into machine code and packages it into an **object file**. Object files contain binary code that isn't yet ready to run—it needs to be linked with other object files and libraries.

Understanding assembly and object files is crucial for:

- Debugging at the machine code level
- Understanding how code maps to hardware
- Analyzing performance bottlenecks
- Reverse engineering and security analysis

In this chapter, we'll explore assembly language syntax and the structure of object files.

5.2 Assembly Language Basics

5.2.1 What is Assembly Language?

Assembly language is a human-readable representation of machine code. Each assembly instruction corresponds to one or more machine instructions. Unlike C, assembly:

- Is architecture-specific (x86-64 assembly ≠ ARM assembly)
- Has a nearly 1-to-1 mapping with machine code
- Provides direct access to registers and memory
- Offers no type safety or high-level abstractions

5.2.2 Assembly Structure

A typical assembly file has three types of content:

```
# Comments start with # (AT&T syntax)

.section .text # Directive: tells assembler about section
.globl main # Directive: make 'main' visible externally
```

```
main: # Label: marks a memory address
   pushq %rbp # Instruction: push rbp onto stack
   movq %rsp, %rbp # Instruction: copy rsp to rbp
   movl $42, %eax # Instruction: load 42 into eax
   popq %rbp # Instruction: restore rbp
   ret # Instruction: return from function
```

Directives: Commands to the assembler (not translated to machine code)
Labels: Named addresses for code or data locations
Instructions: Actual machine instructions

5.3 AT&T vs. Intel Syntax

There are two main assembly syntaxes, and understanding both is important:

5.3.1 AT&T Syntax (GCC default on Linux)

```
# AT&T syntax
movl $5, %eax # Move immediate 5 into eax
movl %eax, -4(%rbp) # Move eax to memory at rbp-4
movl (%rdi), %eax # Move memory at rdi into eax
addl %ecx, %eax # Add ecx to eax (eax = eax + ecx)
```

AT&T syntax features:

- **Source first, destination second**: `mov src, dst`
- **Register prefix**: `%` before register names
- **Immediate prefix**: `$` before constants
- **Size suffix**: `b` (byte), `w` (word), `l` (long/dword), `q` (quadword)
- **Memory syntax**: `disp(base, index, scale)` → `address = base + index*scale + disp`

5.3.2 Intel Syntax (MASM, NASM, Windows)

```
# Intel syntax
mov eax, 5 # Move immediate 5 into eax
mov dword ptr [rbp-4], eax # Move eax to memory at rbp-4
mov eax, [rdi] # Move memory at rdi into eax
add eax, ecx # Add ecx to eax
```

Intel syntax features:

- **Destination first, source second**: `mov dst, src`
- **No register prefix**: `eax` not `%eax`
- **No immediate prefix**: `5` not `$5`

- **Size specifier**: `byte ptr`, `word ptr`, `dword ptr`, `qword ptr`
- **Memory syntax**: `[base + index*scale + disp]`

> **Tip**
>
> Most Linux tools default to AT&T syntax. If you learned assembly on Windows or from tutorials, you might be more familiar with Intel syntax. Both are equivalent—it's just a matter of preference.

5.4 Common x86-64 Instructions

5.4.1 Data Movement

```
# Move data
movl $10, %eax # eax = 10 (immediate to register)
movl %eax, %ecx # ecx = eax (register to register)
movl %eax, (%rdi) # *rdi = eax (register to memory)
movl (%rdi), %eax # eax = *rdi (memory to register)

# Move with zero extension
movzbw %al, %ax # Zero-extend byte to word
movzbl %al, %eax # Zero-extend byte to long

# Move with sign extension
movsbw %al, %ax # Sign-extend byte to word
movsbl %al, %eax # Sign-extend byte to long
movslq %eax, %rax # Sign-extend long to quad

# Exchange
xchg %rax, %rbx # Swap rax and rbx
```

5.4.2 Arithmetic Operations

```
# Addition and subtraction
addl $5, %eax # eax = eax + 5
subl $3, %eax # eax = eax - 3
incl %eax # eax = eax + 1
decl %eax # eax = eax - 1

# Multiplication
imull %ecx # edx:eax = eax * ecx (signed)
mull %ecx # edx:eax = eax * ecx (unsigned)

# Division
idivl %ecx # eax = edx:eax / ecx, edx = remainder (signed)
divl %ecx # eax = edx:eax / ecx, edx = remainder (unsigned)

# Negation
negl %eax # eax = -eax
```

5.4.3 Control Flow

```
# Unconditional jump
jmp label # Jump to label

# Conditional jumps (after cmp or test)
je label # Jump if equal (ZF=1)
jne label # Jump if not equal (ZF=0)
jg label # Jump if greater (signed)
jge label # Jump if greater or equal (signed)
jl label # Jump if less (signed)
jle label # Jump if less or equal (signed)
ja label # Jump if above (unsigned)
jb label # Jump if below (unsigned)

# Function calls
call function # Push return address, jump to function
ret # Pop return address, jump to it
```

5.5 Assembly Directives

5.5.1 Section Directives

```
.section .text # Code section
.section .data # Initialized data section
.section .bss # Uninitialized data section
.section .rodata # Read-only data section
.section .eh_frame # Exception handling information
```

5.5.2 Data Definition Directives

```
.data

# Define bytes
byte_val: .byte 42 # One byte initialized to 42
bytes: .byte 1, 2, 3, 4, 5 # Multiple bytes

# Define longs (4 bytes)
long_val: .long 100000 # 4-byte value
longs: .long 10, 20, 30 # Array of longs

# Define quads (8 bytes)
quad_val: .quad 0x123456789ABCDEF0 # 8-byte value
ptr: .quad string_label # Pointer to string

# Define strings
string_label: .asciz "Hello, World!\n" # Null-terminated string

# Reserve space (BSS section)
.bss
buffer: .skip 1024 # Reserve 1024 bytes (zero-initialized)
```

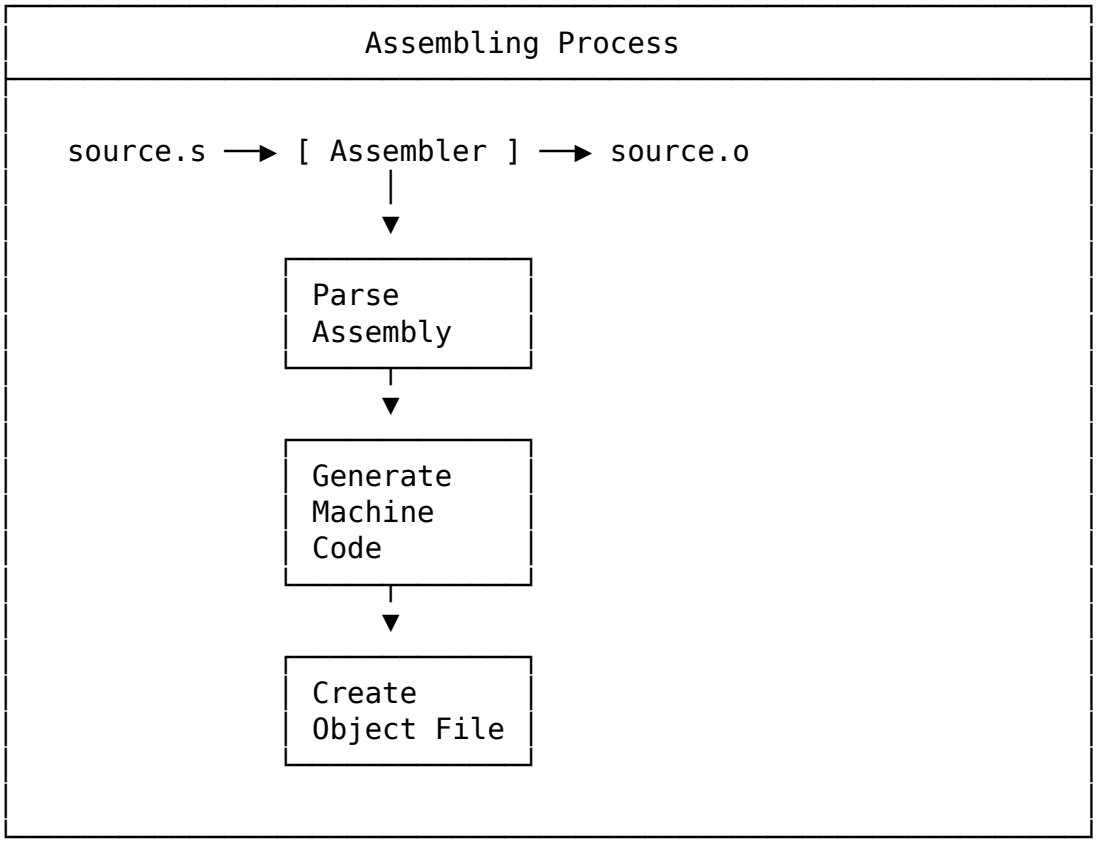
5.5.3 Symbol Directives

```
.globl main # Make 'main' visible to linker
.extern printf # Declare external symbol (often implicit)
.local helper # Keep 'helper' local to this file
.weak weak_symbol # Declare weak symbol
```

5.6 The Assembler

5.6.1 The Assembling Process

The assembler (as on Linux) translates assembly to object files:



5.6.2 Using the Assembler

```
# Assemble source.s to source.o
as source.s -o source.o

# Or using gcc (which calls as)
gcc -c source.s -o source.o

# Generate assembly from C
gcc -S source.c -o source.s
```

```
# Assemble with debugging info
as --gdwarf-2 source.s -o source.o
```

5.7 Object File Structure

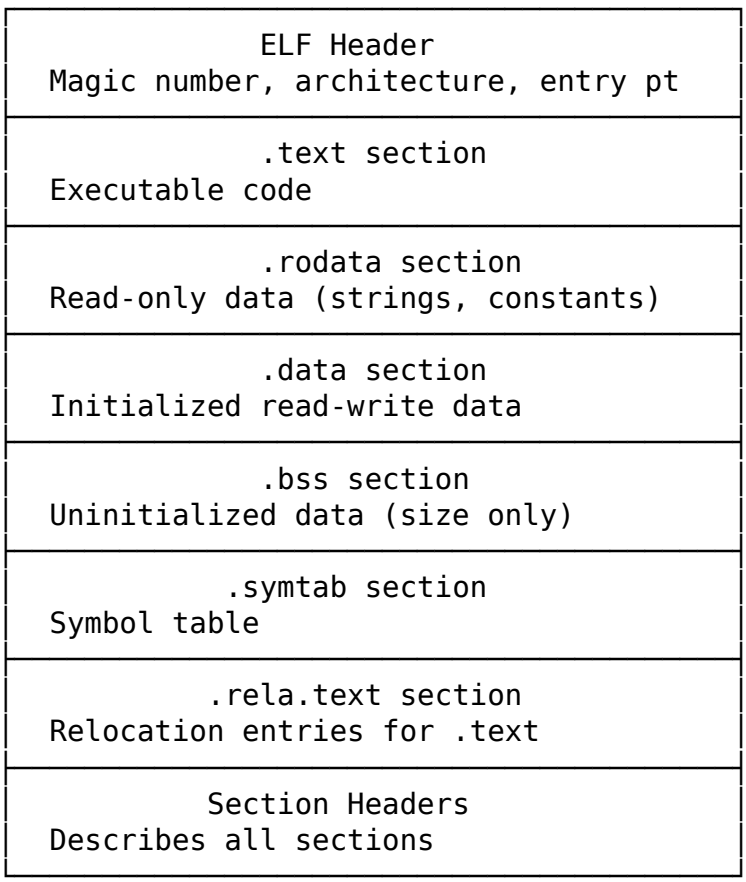
5.7.1 What is an Object File?

An object file (`.o` on Unix, `.obj` on Windows) contains:

- **Machine code**: Binary instructions
- **Data**: Initialized and uninitialized variables
- **Symbol table**: Names and locations of functions and variables
- **Relocation entries**: Locations that need fixing during linking
- **Debug information**: Source line mappings (if compiled with `-g`)

5.7.2 Object File Format

On Linux, object files use the **ELF** (Executable and Linkable Format) format. We'll cover ELF in detail in Chapter 5, but here's the basic structure:



5.7.3 Examining Object Files

```
# View all sections
objdump -h source.o

# View symbol table
nm source.o

# View detailed symbol info
readelf -s source.o

# Disassemble code sections
objdump -d source.o

# View all content
readelf -a source.o
```

5.8 Symbols and Symbol Tables

5.8.1 What are Symbols?

Symbols are named locations in the object file. They include:

- **Defined functions**: Functions implemented in this file
- **Defined variables**: Global and static variables
- **Referenced symbols**: External functions and variables used

5.8.2 Symbol Types

```
$ nm source.o

# Output format: address type name
0000000000000000 T main # T = text (code) section, global
0000000000000000 t helper # t = text section, local
0000000000000010 D global_var # D = data section, initialized
0000000000000000 B bss_var # B = BSS section, uninitialized
                 U printf # U = undefined (external)
```

Common symbol types:

Type	Meaning
T/t	Text (code) section
D/d	Data section (initialized)
B/b	BSS section (uninitialized)
R/r	Read-only data section
U	Undefined (external reference)
C	Common symbol

Uppercase = global/external, lowercase = local

5.9 Relocations

5.9.1 What are Relocations?

Relocations are ”fix-ups” that the linker must perform. When the assembler encounters a reference to an external symbol or a location that’s not yet known, it creates a relocation entry.

```
# Assembly with external reference
call printf # Assembler doesn't know where printf is

# Object file contains:
# - Partial instruction encoding
# - Relocation entry: "fix this location with printf's address"
```

5.9.2 Viewing Relocations

```
# View relocations
objdump -r source.o

# Or with readelf
readelf -r source.o
```

Example output:

```
RELOCATION RECORDS FOR [.text]:
OFFSET           TYPE              VALUE
0000000000000005 R_X86_64_PC32     .rodata-0x0000000000000004
000000000000000a R_X86_64_PLT32    printf-0x0000000000000004
```

5.10 Practical Example: From C to Object File

5.10.1 Source Code

```
// example.c
#include <stdio.h>

int global_var = 42;
static int local_var = 10;

int add(int a, int b) {
   return a + b;
}

static int multiply(int a, int b) {
   return a * b;
}

int main(int argc, char** argv) {
   int sum = add(global_var, local_var);
   printf("Sum: %d\n", sum);
```

```
    return 0;
}
```

5.10.2 Examine Object File

```
gcc -c example.c -o example.o
nm example.o
```

Output:

```
0000000000000000 T add           # Global function
0000000000000000 D global_var    # Global variable
                 U printf        # External reference
0000000000000000 T main          # Global function
0000000000000014 t multiply      # Local function (lowercase t)
0000000000000004 d local_var     # Local variable (lowercase d)
```

5.11 Debug Information

5.11.1 Compiling with Debug Info

```
# Include debug symbols
gcc -g source.c -o program

# Debug info level
gcc -g1 source.c # Minimal
gcc -g2 source.c # Default
gcc -g3 source.c # Maximum (includes macros)

# Debug format
gcc -gdwarf-4 source.c # DWARF version 4
```

5.11.2 Viewing Debug Info

```
# View debug sections
readelf --debug-dump=info program

# View line info
readelf --debug-dump=line program

# Use addr2line
addr2line -e program 0x401126 # What source line is at this address?

# Use objdump with source
objdump -S program # Interleave source with assembly
```

5.12 Key Takeaways

1. **Assembly is architecture-specific**: x86-64 assembly won't work on ARM, and vice versa.
2. **Two syntax styles**: AT&T (Linux default) and Intel (Windows, NASM). Learn to read both.
3. **Instructions, directives, and labels**: These are the three building blocks of assembly files.
4. **Object files are intermediate products**: They contain machine code but need linking before execution.
5. **Symbols name locations**: The symbol table maps names to addresses within the object file.
6. **Relocations mark fix-ups**: The linker fills in addresses that the assembler couldn't determine.
7. **Debug info maps machine to source**: DWARF format connects binary addresses to source lines.
8. **Tools for inspection**: `objdump`, `readelf`, `nm`, and `addr2line` are essential for analyzing object files.

5.13 Looking Ahead

Now that we understand object files, we'll dive deeper into the ELF format in Chapter 5. We'll examine every field in the ELF header, section headers, and program headers, giving you complete mastery over binary file analysis.

Chapter 6

ELF Format Deep Dive

6.1 Introduction

The **Executable and Linkable Format (ELF)** is the standard binary format on Unix-like systems. Whether you're dealing with object files, executables, shared libraries, or core dumps, they all use ELF. Understanding ELF gives you the power to analyze binaries, debug linking problems, and understand how programs are loaded.

In this chapter, we'll dissect ELF files byte by byte, exploring every header, table, and section in detail.

6.2 ELF Overview

6.2.1 What is ELF?

ELF is a flexible binary format that supports:

- **Executables**: Programs ready to run
- **Object files**: Intermediate compilation outputs
- **Shared libraries**: Dynamically loaded code (`.so` files)
- **Core dumps**: Memory snapshots for debugging

6.2.2 Two Views of ELF

ELF files can be viewed from two perspectives:

Linking View (used by linkers):

- Sections: `.text`, `.data`, `.bss`, `.symtab`, etc.
- Section header table describes all sections

Execution View (used by the loader):

- Segments: Groups of sections loaded into memory
- Program header table describes all segments

6.3 The ELF Header

6.3.1 ELF Header Structure

The ELF header sits at the very beginning of the file and contains essential identification information:

```
// From /usr/include/elf.h
#define EI_NIDENT 16

typedef struct {
   unsigned char e_ident[EI_NIDENT]; // Identification bytes
   uint16_t e_type; // Object file type
   uint16_t e_machine; // Architecture
   uint32_t e_version; // Object file version
   uint64_t e_entry; // Entry point address
   uint64_t e_phoff; // Program header table offset
   uint64_t e_shoff; // Section header table offset
   uint32_t e_flags; // Processor-specific flags
   uint16_t e_ehsize; // ELF header size
   uint16_t e_phentsize; // Program header table entry size
   uint16_t e_phnum; // Program header table entry count
   uint16_t e_shentsize; // Section header table entry size
   uint16_t e_shnum; // Section header table entry count
   uint16_t e_shstrndx; // Section name string table index
} Elf64_Ehdr;
```

6.3.2 The e_ident Array (Magic Bytes)

The first 16 bytes identify the file:

Offset	Size	Description
0	4	Magic number: `0x7f 'E' 'L' 'F'`
4	1	32-bit (1) or 64-bit (2)
5	1	Endianness: little (1) or big (2)
6	1	ELF version (always 1)
7	1	OS/ABI identification
8-15	8	Padding (unused)

Magic Number: The first four bytes are always `0x7f ELF` (or `7f 45 4c 46` in hex).

Class (32-bit vs 64-bit):

```
ELFCLASS32 = 1  // 32-bit architecture
ELFCLASS64 = 2  // 64-bit architecture
```

6.3.3 e_type: File Type

Value	Name	Description
0	ET_NONE	No file type
1	ET_REL	Relocatable (object file)
2	ET_EXEC	Executable
3	ET_DYN	Shared object (library or PIE executable)
4	ET_CORE	Core dump

6.3.4 e_machine: Architecture

Value	Name	Architecture
3	EM_386	x86 (32-bit)
62	EM_X86_64	x86-64
40	EM_ARM	ARM
183	EM_AARCH64	ARM64
243	EM_RISCV	RISC-V

6.3.5 Viewing the ELF Header

```
# View the ELF header
readelf -h program

# Or just the identification
file program
```

6.4 Section Headers

6.4.1 Section Header Structure

Sections contain the actual code and data. The section header table is an array of section headers:

```
typedef struct {
   uint32_t sh_name; // Section name (index into string table)
   uint32_t sh_type; // Section type
   uint64_t sh_flags; // Section flags
   uint64_t sh_addr; // Virtual address in memory
   uint64_t sh_offset; // Offset in file
   uint64_t sh_size; // Size in bytes
   uint32_t sh_link; // Link to another section
   uint32_t sh_info; // Additional info
   uint64_t sh_addralign; // Alignment
   uint64_t sh_entsize; // Entry size (if section holds table)
} Elf64_Shdr;
```

6.4.2 Common Section Types (sh_type)

Value	Name	Description
0	SHT_NULL	Inactive header
1	SHT_PROGBITS	Program-defined contents (code, data)
2	SHT_SYMTAB	Symbol table
3	SHT_STRTAB	String table
4	SHT_RELA	Relocation entries with addends
5	SHT_HASH	Symbol hash table
6	SHT_DYNAMIC	Dynamic linking info
8	SHT_NOBITS	No space in file (BSS)

6.4.3 Section Flags (sh_flags)

Flag	Value	Description
SHF_WRITE	0x1	Writable
SHF_ALLOC	0x2	Occupies memory during execution
SHF_EXECINSTR	0x4	Executable code

6.4.4 Standard Sections

Common ELF Sections

Section	Flags	Content
.text	AX	Executable code
.rodata	A	Read-only data (const, strings)
.data	WA	Initialized writable data
.bss	WA	Uninitialized data (zero-filled)
.symtab	-	Symbol table
.strtab	-	String table for symbol names
.shstrtab	-	Section name string table
.rela.text	-	Relocations for .text
.dynamic	WA	Dynamic linking information
.dynsym	A	Dynamic symbol table
.got	WA	Global Offset Table
.plt	AX	Procedure Linkage Table
.eh_frame	A	Exception handling info
.debug_*	-	DWARF debug information

A = SHF_ALLOC, W = SHF_WRITE, X = SHF_EXECINSTR

6.4.5 Viewing Sections

```
# List all sections
readelf -S program

# Or with objdump
objdump -h program

# View section contents (hex dump)
objdump -s -j .rodata program
```

6.5 Program Headers

6.5.1 Program Header Structure

Program headers describe **segments**—contiguous chunks of the file that the loader maps into memory:

```
typedef struct {
   uint32_t p_type; // Segment type
   uint32_t p_flags; // Segment flags
   uint64_t p_offset; // Offset in file
   uint64_t p_vaddr; // Virtual address in memory
   uint64_t p_paddr; // Physical address (unused)
   uint64_t p_filesz; // Size in file
   uint64_t p_memsz; // Size in memory
   uint64_t p_align; // Alignment
} Elf64_Phdr;
```

6.5.2 Segment Types (p_type)

Value	Name	Description
0	PT_NULL	Inactive
1	PT_LOAD	Loadable segment
2	PT_DYNAMIC	Dynamic linking info
3	PT_INTERP	Interpreter path
6	PT_PHDR	Program header table itself
0x6474e551	PT_GNU_STACK	Stack executability
0x6474e552	PT_GNU_RELRO	Read-only after relocation

6.5.3 Segment Flags (p_flags)

Flag	Value	Description
PF_X	0x1	Executable
PF_W	0x2	Writable
PF_R	0x4	Readable

6.5.4 Key Segments

PT_LOAD: The most important segment type. The loader maps these into memory:

- `.text` usually forms one LOAD segment (RX - read+execute)
- `.data` and `.bss` form another (RW - read+write)

PT_INTERP: Contains the path to the dynamic linker:

```
/lib64/ld-linux-x86-64.so.2
```

PT_GNU_STACK: Controls whether the stack is executable (security feature).
PT_GNU_RELRO: Marks regions to become read-only after relocation (security feature).

6.5.5 Viewing Program Headers

```
# View program headers
readelf -l program
```

6.6 Symbol Tables

6.6.1 Symbol Table Entry Structure

```
typedef struct {
   uint32_t st_name; // Symbol name (string table index)
   uint8_t st_info; // Type and binding
   uint8_t st_other; // Visibility
   uint16_t st_shndx; // Section index
   uint64_t st_value; // Symbol value (address)
   uint64_t st_size; // Symbol size
} Elf64_Sym;
```

6.6.2 Symbol Binding (st_info upper 4 bits)

Value	Name	Description
0	STB_LOCAL	Local to this file
1	STB_GLOBAL	Visible to all files
2	STB_WEAK	Like global, but lower precedence

6.6.3 Viewing Symbols

```
# View symbol table
readelf -s program
nm program

# View dynamic symbols only
readelf --dyn-syms program
```

6.7 Relocations

6.7.1 Relocation Entry Structure

```
// Relocation with addend
typedef struct {
   uint64_t r_offset; // Where to apply relocation
   uint64_t r_info; // Symbol index and type
   int64_t r_addend; // Constant addend
} Elf64_Rela;
```

6.7.2 Common x86-64 Relocation Types

Name	Value	Description
R_X86_64_64	1	64-bit absolute
R_X86_64_PC32	2	32-bit PC-relative
R_X86_64_GOT32	3	32-bit GOT offset
R_X86_64_PLT32	4	32-bit PLT-relative
R_X86_64_GOTPCREL	9	GOT + PC-relative

6.7.3 Viewing Relocations

```
# View relocations
readelf -r object.o
objdump -r object.o
```

6.8 Dynamic Linking Structures

6.8.1 The .dynamic Section

The `.dynamic` section contains an array of entries for the dynamic linker:

```
typedef struct {
   int64_t d_tag; // Type
   union {
      uint64_t d_val; // Integer value
      uint64_t d_ptr; // Address value
   } d_un;
} Elf64_Dyn;
```

6.8.2 Global Offset Table (GOT)

The GOT contains addresses of external symbols, filled in by the dynamic linker.

6.8.3 Procedure Linkage Table (PLT)

The PLT contains stubs for calling external functions.

6.8.4 Viewing Dynamic Info

```
# View dynamic section
readelf -d program

# View needed libraries
ldd program
```

6.9 Practical ELF Analysis

6.9.1 Identifying File Types

```
file program
# Output: program: ELF 64-bit LSB shared object, x86-64, ...
```

6.9.2 Finding Entry Point

```
readelf -h program | grep Entry
# Output: Entry point address: 0x1060
```

6.9.3 Finding String Constants

```
strings program
strings -t x program # With hex offset
```

6.9.4 Checking Security Features

```
# Check for stack executable (NX bit)
readelf -l program | grep GNU_STACK

# Check for RELRO
readelf -l program | grep GNU_RELRO

# Check for PIE (position-independent)
file program | grep shared
readelf -h program | grep Type
```

6.10 ELF Tools Summary

Tool	Purpose
`readelf`	Comprehensive ELF analysis
`objdump`	Disassembly and section viewing
`nm`	Symbol table viewing
`strings`	Extract string constants
`strip`	Remove symbols
`ldd`	Show library dependencies
`file`	Quick file type identification

6.11 Key Takeaways

1. **ELF has two views**: Linking view (sections) for linkers, execution view (segments) for loaders.
2. **The ELF header is the roadmap**: It points to the program header table and section header table.
3. **Sections contain code and data**: `.text` for code, `.data` for initialized data, `.bss` for uninitialized data.
4. **Program headers describe memory mapping**: PT_LOAD segments are loaded into memory by the loader.
5. **Symbols name locations**: The symbol table maps names to addresses and contains binding/type information.
6. **Relocations enable linking**: They tell the linker where to fill in final addresses.
7. **Dynamic structures enable runtime linking**: GOT, PLT, and .dynamic work together for shared library support.
8. **Tools are your friends**: `readelf`, `objdump`, and `nm` reveal everything about ELF files.

6.12 Looking Ahead

Now that we understand the ELF format, we can explore how the linker uses this information to combine object files into executables. In Chapter 6, we'll cover symbol resolution, relocation application, and the complete linking process.

Chapter 7

Linking and Relocation

7.1 Introduction

The **linker** is the final stage of the build process, combining object files and libraries into a single executable or library. It resolves symbol references, applies relocations, and produces the final binary that can be loaded and executed.

Understanding linking helps you:

- Debug ”undefined symbol” and ”multiple definition” errors
- Organize code across multiple files effectively
- Understand static vs. dynamic linking tradeoffs
- Optimize build times and binary sizes

In this chapter, we’ll explore the complete linking process from object files to executable.

7.2 What is Linking?

7.2.1 The Linker’s Role

When you compile multiple source files, each produces a separate object file. These object files are incomplete—they contain references to symbols defined in other files. The linker’s job is to:

1. **Collect**: Gather all object files and libraries
2. **Resolve**: Match symbol references with definitions
3. **Relocate**: Adjust addresses based on final layout
4. **Output**: Produce a single executable or library

7.2.2 Why Separate Linking?

You might wonder: why not compile everything together? Separation provides:

- **Incremental builds**: Only recompile changed files
- **Separate compilation**: Teams can work independently

- **Code reuse**: Libraries can be linked into many programs
- **Modularity**: Clear interfaces between components

7.3 Symbol Resolution

7.3.1 The Symbol Resolution Problem

Each object file has:

- **Defined symbols**: Functions and variables it provides
- **Undefined symbols**: References to external functions/variables

The linker must ensure every undefined symbol has exactly one definition.

7.3.2 Symbol Types and Precedence

Strong Symbols: Functions, initialized global variables
Weak Symbols: Symbols declared with `__attribute__((weak))`
Resolution rules:

1. Multiple strong symbols with same name → **error**
2. One strong symbol, multiple weak symbols → **use strong**
3. Multiple weak symbols → **arbitrary choice**

```
// file1.c
int global_var = 10; // Strong symbol
void foo(void) { } // Strong symbol

// file2.c
int global_var; // Weak symbol (tentative definition)
__attribute__((weak)) void foo(void) { } // Weak symbol

// Linker chooses file1.c's definitions (strong wins)
```

7.3.3 Common Linker Errors

Undefined Symbol:

```
/usr/bin/ld: /tmp/ccXXXX.o: in function `main':
main.c:(.text+0x15): undefined reference to `missing_function'
collect2: error: ld returned 1 exit status
```

Cause: Referenced symbol not defined in any linked file or library.
Multiple Definition:

```
/usr/bin/ld: /tmp/ccYYYY.o:(.data+0x0): multiple definition of `global_var'
/usr/bin/ld: /tmp/ccXXXX.o:(.data+0x0): first defined here
collect2: error: ld returned 1 exit status
```

Cause: Symbol defined in multiple object files.
Solutions:

- For undefined: Add the defining file/library to link command
- For multiple: Use `static` for file-local variables, or `extern` declarations

7.4 Relocation

7.4.1 What is Relocation?

Object files are compiled as if they start at address 0. The linker must adjust all addresses to reflect the final memory layout. This process is **relocation**.

7.4.2 Relocation Types

Different relocation types specify how to compute the final value:
Absolute Relocation: Store the symbol's actual address

```
R_X86_64_64: Store full 64-bit address
```

PC-Relative Relocation: Store the offset from current instruction

```
R_X86_64_PC32: Store (symbol_address - current_address)
R_X86_64_PLT32: Store (PLT_entry - current_address)
```

GOT-Relative: Store offset to GOT entry

```
R_X86_64_GOTPCREL: Store (GOT_entry - current_address)
```

7.4.3 Relocation Process

For each relocation entry in object files:

1. Find the target symbol's final address
2. Compute the relocation value based on type
3. Write the value to the specified offset in the output

7.4.4 Viewing Relocations

```
# In object files (before linking)
readelf -r object.o

# In final executable (fewer, for dynamic linking)
readelf -r executable
```

7.5 The Linking Process in Detail

7.5.1 Linker Input Processing

The linker processes inputs in order:

```
gcc main.o utils.o -lm -o program
```

Order matters for static libraries:

```
1. Process main.o: collect undefined symbols (utils functions)
2. Process utils.o: provides utils functions, may add undefined symbols
3. Process -lm (libm.a): only extract needed objects
```

Critical rule: Static libraries should come after files that use them:

```
# Wrong: linker hasn't seen the undefined symbols yet
gcc -lm main.o -o program # May fail to find math functions

# Correct: main.o's undefined symbols trigger libm extraction
gcc main.o -lm -o program
```

7.5.2 Section Merging

The linker combines sections from all inputs:

```
Input A:        Input B:        Output:
.text (100)     .text (200)     .text (300)
.rodata (50)    .rodata (30)    .rodata (80)
.data (20)      .data (40)      .data (60)
.bss (10)       .bss (15)       .bss (25)
```

Sections are merged by name and type, maintaining alignment requirements.

7.5.3 Address Assignment

The linker assigns virtual addresses to all sections:

```
Default layout (simplified):
0x400000: .text (code)
0x600000: .rodata (read-only data)
0x800000: .data (initialized data)
0x800XXX: .bss (uninitialized data)
```

The actual layout is determined by:

- Linker script (default or custom)
- Section alignment requirements
- Memory protection constraints

7.6 Static Linking

7.6.1 What is Static Linking?

Static linking embeds library code directly into the executable:

```
# Create a static library
ar rcs libmylib.a file1.o file2.o file3.o

# Link statically with the library
gcc main.o -L. -lmylib -o program

# Or link completely static
gcc -static main.o -o program
```

7.6.2 Static Library Format

Static libraries (`.a` files) are **archives** containing multiple object files:

```
libmylib.a:
├── file1.o
├── file2.o
└── file3.o

# View contents
ar -t libmylib.a
ar -x libmylib.a   # Extract all .o files
```

7.6.3 Selective Linking

The linker only extracts objects that satisfy undefined symbols:

```
libmylib.a:
├── string_utils.o (defines: str_lower, str_upper)
├── math_utils.o   (defines: add, subtract)
└── file_utils.o   (defines: read_file, write_file)

main.o references: str_lower, add

Linker extracts: string_utils.o, math_utils.o
Ignores: file_utils.o (no referenced symbols)
```

This prevents unnecessary code bloat.

7.6.4 Pros and Cons of Static Linking

Advantages:

- Self-contained executable (no external dependencies)
- Predictable behavior (library version is fixed)

- No runtime linking overhead

Disadvantages:

- Larger executables
- Memory inefficiency (each program has its own copy)
- Updates require recompilation
- Security updates harder to deploy

7.7 Dynamic Linking

7.7.1 What is Dynamic Linking?

Dynamic linking defers library resolution to runtime:

```
# Create a shared library
gcc -shared -fPIC -o libmylib.so file1.c file2.c

# Link against shared library
gcc main.c -L. -lmylib -o program

# At runtime
LD_LIBRARY_PATH=. ./program
```

7.7.2 Position-Independent Code (PIC)

Shared libraries must be position-independent—they can load at any address:

```
# Compile with -fPIC
gcc -fPIC -c file.c -o file.o

# Create shared library
gcc -shared file.o -o libfile.so
```

PIC uses relative addressing and the GOT.

7.7.3 The Dynamic Linker

The dynamic linker (`ld.so`) runs when the program starts:

1. Read the program's dynamic section
2. Find needed libraries (DT_NEEDED entries)
3. Load libraries into memory
4. Resolve symbols (lazily or eagerly)
5. Update GOT entries

```
# View dynamic dependencies
ldd program

# Trace dynamic linking
LD_DEBUG=libs ./program
LD_DEBUG=symbols ./program
```

7.7.4 Lazy vs. Eager Binding

Lazy Binding (default): Resolve symbols on first use

- Faster startup
- Unused symbols never resolved
- Implemented via PLT

Eager Binding: Resolve all symbols at startup

```
LD_BIND_NOW=1 ./program
```

- Slower startup
- All symbols guaranteed to exist
- More predictable behavior

7.7.5 Pros and Cons of Dynamic Linking

Advantages:

- Smaller executables
- Memory efficiency (shared code pages)
- Easy updates (replace library file)
- Runtime plugin support

Disadvantages:

- Runtime overhead
- Dependency management (”DLL hell”)
- Security concerns (LD_PRELOAD attacks)
- Version compatibility issues

7.8 Linker Scripts

7.8.1 What is a Linker Script?

Linker scripts control exactly how sections are laid out:

```
/* Simple linker script */
ENTRY(main)

SECTIONS
{
   . = 0x400000; /* Start address */

   .text : {
      *(.text) /* All .text sections */
   }

   . = 0x600000;

   .rodata : {
      *(.rodata)
   }

   . = 0x800000;

   .data : {
      *(.data)
   }

   .bss : {
      *(.bss)
   }
}
```

7.8.2 Using Linker Scripts

```
# Use custom linker script
ld -T custom.ld object.o -o program

# Or with gcc
gcc -T custom.ld object.o -o program
```

7.8.3 View Default Linker Script

```
# See the default script
ld --verbose

# For gcc's default
gcc -Wl,--verbose 2>&1 | grep -A1000 "^======="
```

7.9 Common Linker Flags

7.9.1 Library Search Paths

```
# Add library search path
gcc -L/path/to/libs program.c -lmylib

# Add runtime library path
gcc -Wl,-rpath,/path/to/libs program.c -lmylib

# Add both compile-time and runtime paths
gcc -L/path/to/libs -Wl,-rpath,/path/to/libs program.c -lmylib
```

7.9.2 Optimization Flags

```
# Remove unused sections
gcc -ffunction-sections -fdata-sections -Wl,--gc-sections

# Link-time optimization (LTO)
gcc -flto program.c

# Dead code elimination
gcc -Wl,--as-needed # Only link needed libraries
```

7.9.3 Security Flags

```
# Enable RELRO (read-only relocations)
gcc -Wl,-z,relro program.c

# Full RELRO (eager binding + read-only GOT)
gcc -Wl,-z,relro,-z,now program.c

# Non-executable stack
gcc -Wl,-z,noexecstack program.c
```

7.10 Linker Errors and Solutions

7.10.1 Undefined Reference

`undefined reference to `symbol_name'`

Causes:

- Forgot to link a library
- Misspelled symbol name
- Symbol is static in another file
- C++ name mangling mismatch

Solutions:

```
# Add missing library
gcc program.c -lmissinglib

# For C++ code, use extern "C"
extern "C" void c_function();

# Check if symbol exists
nm library.a | grep symbol_name
```

7.10.2 Multiple Definition

```
multiple definition of `symbol_name'
```

Causes:

- Symbol defined in multiple files
- Header defines variable (not just declares)

Solutions:

```
// In header: declare only
extern int global_var;

// In one source file: define
int global_var = 10;

// Or use static for file-local
static int file_local_var = 10;
```

7.10.3 Cannot Find Library

```
cannot find -lmylib
```

Solutions:

```
# Add library path
gcc -L/path/to/lib program.c -lmylib

# Check library exists
find /usr -name "libmylib*"

# Check ldconfig cache
ldconfig -p | grep mylib
```

7.11 Key Takeaways

1. **Linkers combine object files**: They resolve symbols and apply relocations.
2. **Symbol resolution matches references to definitions**: Strong symbols override weak ones; duplicates cause errors.

3. **Relocation adjusts addresses**: Object files start at 0; linker computes final addresses.
4. **Static linking embeds libraries**: Self-contained but larger; updates require recompilation.
5. **Dynamic linking defers to runtime**: Smaller, updatable, but with runtime overhead.
6. **Order matters for static libraries**: Libraries should come after files that use them.
7. **Linker scripts control layout**: They specify section addresses and ordering.
8. **Many linker flags optimize and secure**: LTO, garbage collection, RELRO, etc.

7.12 Looking Ahead

With linking complete, we have an executable ready to run. But where do the standard library functions come from? In Chapter 7, we'll explore static and dynamic libraries in depth, learning how to create, use, and manage them.

Chapter 8

Static and Dynamic Libraries

8.1 Introduction

In Chapter 6, we learned how the linker combines object files into a single executable by resolving symbols and applying relocations. But what happens when you want to reuse code across multiple programs? Must every program contain a complete copy of all the code it uses?

This chapter explores **libraries**—collections of compiled object files that can be linked into multiple programs. We'll examine two fundamentally different approaches: **static libraries** (`.a` files) that are copied into the executable at link time, and **dynamic libraries** (`.so` files) that are loaded at runtime.

Understanding the difference between static and dynamic linking is crucial for:

- **Binary distribution**: Deciding whether to ship dependencies with your application
- **Security updates**: Understanding how library updates (or lack thereof) affect your programs
- **Performance**: Balancing startup time, memory usage, and execution speed
- **Cross-platform compatibility**: Ensuring your code runs across different systems

8.2 The Problem Libraries Solve

Imagine you're developing multiple applications that all need common functionality—say, mathematical operations, string handling, or file I/O. Without libraries, you'd have three bad options:

1. **Copy source code** into each project (maintenance nightmare)
2. **Copy object files** into each project (wastes disk space)
3. **Manually link the right object files** each time (error-prone)

Libraries solve all these problems by providing a packaged collection of object files that can be easily linked into any program.

Example scenario: You've written useful math functions:

```c
// math_utils.c
int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }
int factorial(int n) {
   if (n <= 1) return 1;
   return n * factorial(n - 1);
}
```

You want to use these functions in multiple programs. Let's see how both static and dynamic libraries make this possible.

8.3 Static Libraries (.a files)

8.3.1 What is a Static Library?

A **static library** is simply an archive (collection) of object files, typically with the `.a` extension. When you link against a static library, the linker **extracts** the object files containing the symbols you reference and **copies** them into your executable.

Think of a static library like a ZIP file of object files—the linker can open it, pull out what it needs, and ignore the rest.

Key characteristics:

- **File extension**: `.a` (archive) on Unix-like systems, `.lib` on Windows
- **Link time**: Symbols are resolved at link time (during compilation)
- **Runtime**: No external dependencies; the executable is self-contained
- **Size**: Larger binaries because all needed code is included

8.3.2 Creating a Static Library

Let's create a static library from our math utilities:

```
# Step 1: Compile source files to object files
gcc -c math_utils.c -o math_utils.o

# Step 2: Create a static library using ar (archiver)
ar rcs libmathutils.a math_utils.o
```

The `ar` command options:

- `r`: Insert (replace) files into the archive
- `c`: Create the archive if it doesn't exist
- `s`: Write an object file index into the archive

The resulting `libmathutils.a` contains the compiled code from `math_utils.o`. The `lib` prefix and `.a` extension are conventions—the linker expects libraries to follow this naming pattern.

Multiple object files in one library:

```
# Compile multiple source files
gcc -c math_utils.c -o math_utils.o
gcc -c string_utils.c -o string_utils.o
gcc -c file_utils.c -o file_utils.o

# Create a library with all of them
ar rcs libutils.a math_utils.o string_utils.o file_utils.o
```

Now `libutils.a` contains three object files that can be selectively linked.

8.3.3 Using a Static Library

To use a static library in your program:

```
// main.c
#include <stdio.h>

// Forward declarations (typically in a header file)
int add(int a, int b);
int multiply(int a, int b);
int factorial(int n);

int main(void) {
   printf("5 + 3 = %d\n", add(5, 3));
   printf("5 * 3 = %d\n", multiply(5, 3));
   printf("5! = %d\n", factorial(5));
   return 0;
}
```

Compile and link with the static library:

```
gcc main.c -o main -L. -lmathutils
```

The linker flags:

- `-L.`: Add the current directory (`.`) to the library search path
- `-lmathutils`: Link against `libmathutils.a` (the `lib` prefix and `.a` extension are implied)

What the linker does:

1. Reads `main.o` and finds undefined symbols: `add`, `multiply`, `factorial`
2. Searches libraries in order for these symbols
3. Finds them in `libmathutils.a`
4. Extracts the necessary object files from the archive
5. Copies the code into the final executable
6. Resolves all relocations

The resulting `main` executable is completely self-contained—it needs no external files to run.

8.3.4 Static Library Internals: The Symbol Index

When we created the archive with `ar rcs`, the `s` option created a **symbol index** that makes linking much faster. Without this index, the linker would have to scan every object file in the archive to find symbols. With the index, the linker can quickly look up which object file contains each needed symbol.

Let's examine the archive structure:

```
# List the contents of the archive
ar t libmathutils.a

# Display the symbol table (index)
nm -s libmathutils.a

# Or use objdump to see detailed information
objdump -a libmathutils.a
```

Typical output:

```
Archive index:

add in math_utils.o
multiply in math_utils.o
factorial in math_utils.o
```

This index tells the linker exactly which object file contains each symbol.

8.3.5 The "Extract Only What's Needed" Rule

A crucial property of static linking: **the linker only extracts object files that contain referenced symbols**.

Example: If `libutils.a` contains `math_utils.o`, `string_utils.o`, and `file_utils.o`, and your program only calls functions from `math_utils.o`, then only that object file is included in your executable.

```
// main2.c - only uses math functions
int main(void) {
   return add(1, 2); // Only references 'add'
}
```

Linking this program:

```
gcc main2.c -o main2 -L. -lutils
```

The linker:

1. Sees reference to `add`
2. Looks up `add` in the library index
3. Finds `add` in `math_utils.o`
4. Extracts only `math_utils.o` from `libutils.a`
5. Ignores `string_utils.o` and `file_utils.o`

This selective linking keeps binary size manageable.

Warning

If an object file in a static library references symbols from another object file in the same library, the linker will include both. This is why careful library organization matters—group related functions together in the same source file.

8.3.6 Advantages and Disadvantages of Static Libraries

Advantages:

1. **Self-contained executables**: No runtime dependencies; easier to distribute
2. **Predictable behavior**: You know exactly which version of library code you're using
3. **Simpler deployment**: Just copy one file
4. **Faster startup**: No dynamic linking overhead at program start
5. **Better performance**: No position-independent code overhead; more optimization opportunities

Disadvantages:

1. **Larger binaries**: Every program contains its own copy of library code
2. **Disk space waste**: If 10 programs use the same library, that code exists 10 times on disk
3. **Memory inefficiency**: If 10 programs using the same library run simultaneously, the library code occupies memory 10 times
4. **Security updates difficult**: To fix a bug in library code, you must recompile all programs using that library
5. **No sharing**: No way for multiple programs to share library code in memory

8.4 Dynamic Libraries (.so files)

8.4.1 What is a Dynamic Library?

A **dynamic library** (also called a **shared library** or **shared object**) is compiled code that can be loaded by multiple programs at runtime. Instead of copying library code into each executable, the operating system loads the dynamic library once and shares it among all running programs that need it.

Key characteristics:

- **File extension**: `.so` (shared object) on Unix-like systems, `.dll` (dynamic-link library) on Windows, `.dylib` on macOS

- **Link time**: Symbols are resolved at link time, but the library isn't included in the executable
- **Runtime**: The dynamic linker loads the library when the program starts (or on first use)
- **Size**: Smaller binaries because library code is not included

8.4.2 Creating a Dynamic Library

Creating a dynamic library requires special compilation options to ensure the code can be loaded at any memory address:

```
# Compile with Position-Independent Code (PIC) flag
gcc -fPIC -c math_utils.c -o math_utils.o

# Create a shared library
gcc -shared -o libmathutils.so math_utils.o
```

The critical flags:

- `-fPIC`: Generate **position-independent code** (PIC). This allows the code to be loaded at any memory address.
- `-shared`: Tells the linker to produce a shared library instead of an executable

Why Position-Independent Code?

Static libraries are linked at a fixed address known at link time. But dynamic libraries must be capable of being loaded at different addresses in different processes, because the address space layout varies depending on what other libraries are loaded.

PIC achieves this through:

- **Relative addressing**: Using relative jumps and calls instead of absolute addresses
- **GOT (Global Offset Table)**: A table of pointers to external data
- **PLT (Procedure Linkage Table)**: Indirect jumps for external function calls

We'll explore GOT and PLT in detail in section 8.5.

8.4.3 Using a Dynamic Library

Using a dynamic library looks similar to using a static library from the programmer's perspective:

```
gcc main.c -o main -L. -lmathutils
```

But the resulting executable is fundamentally different:

```
# Check the dynamic library dependencies
ldd main
```

Typical output:

```
    linux-vdso.so.1 (0x00007ffc12345000)
    libmathutils.so => ./libmathutils.so (0x00007f8b4a1b2000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f8b49fe0000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f8b4a3b5000)
```

The `ldd` command shows which shared libraries our executable depends on. Notably:

- `libmathutils.so` is listed as a dependency
- The executable doesn't contain the library code—it just has a reference to it

When we run `./main`:

1. The operating system's **dynamic linker** (`ld-linux.so`) runs first
2. It reads the executable's list of needed shared libraries
3. It loads each required library into memory
4. It resolves all symbol references between the executable and libraries
5. Finally, it transfers control to the program's `main` function

8.4.4 Library Search Paths

The dynamic linker needs to find the `.so` files at runtime. It searches in this order:

1. **RPATH/RUNPATH**: Embedded in the executable (specified with `-Wl,-rpath` during compilation)
2. **LD_LIBRARY_PATH**: Environment variable (colon-separated list of directories)
3. **Trusted system directories**: `/lib`, `/usr/lib`, and others configured in `/etc/ld.so.conf`

Example with RPATH:

```
gcc main.c -o main -L. -lmathutils -Wl,-rpath,/path/to/libs
```

Example with LD_LIBRARY_PATH:

```
export LD_LIBRARY_PATH=/path/to/libs:$LD_LIBRARY_PATH
./main
```

For system-wide installation:

```
sudo cp libmathutils.so /usr/local/lib/
sudo ldconfig # Update the linker cache
```

8.5 Dynamic Linking Internals: PLT and GOT

8.5.1 The Problem of Address Resolution

When a dynamically linked program calls a library function, how does it know where that function lives in memory? The address isn't known at compile time because:

1. The library might be loaded at different addresses in different runs (due to ASLR)
2. The library might be updated to a different version

The solution involves two data structures: **PLT** (Procedure Linkage Table) and **GOT** (Global Offset Table).

8.5.2 PLT: Procedure Linkage Table

The PLT is a table of **stubs**—small pieces of code that jump to the actual function. Each external function has an entry in the PLT.

When your code calls a library function like `printf`, it actually jumps to the PLT entry for `printf`.

8.5.3 GOT: Global Offset Table

The GOT is a table of **addresses**—pointers to the actual locations of external symbols. Each PLT entry has a corresponding GOT entry.

8.5.4 Lazy Binding: The First Call

Modern systems use **lazy binding** to optimize startup time. Library functions aren't resolved until they're actually called. Here's what happens on the first call to `printf`:

1. **Your code** calls `printf`
2. **Actually calls** `PLT[printf]` (the PLT entry)
3. **PLT entry** jumps to the address in `GOT[printf]`
4. **Initially**, `GOT[printf]` points back to the PLT resolver
5. **PLT resolver** runs and:
   (a) Identifies which symbol needs resolution (`printf`)
   (b) Asks the dynamic linker to find the real address of `printf` in libc
   (c) Writes that address into `GOT[printf]`
   (d) Jumps to the real `printf`
6. **Future calls** to `printf` jump directly from PLT to the real address via GOT (no resolver needed)

This is called **lazy binding** because resolution happens "lazily"—only when needed.

8.5.5 Observing PLT and GOT

You can examine the PLT and GOT using various tools:

```
# Compile a program that uses library functions
gcc -o demo demo.c -L. -lmathutils

# Examine the PLT
objdump -d -j .plt demo

# Examine the GOT
objdump -d -j .got demo
objdump -d -j .got.plt demo

# See relocation entries (what will be resolved)
readelf -r demo
```

8.6 Static vs Dynamic Libraries: Tradeoffs

8.6.1 Binary Size Comparison

Let's compare executable sizes with actual numbers:

Static linking:

```
gcc -o main_static main.c libmathutils.a
ls -lh main_static
# Output: -rwxr-xr-x 1 user user 872K Jan 10 10:00 main_static
```

Dynamic linking:

```
gcc -o main_dynamic main.c -lmathutils
ls -lh main_dynamic
# Output: -rwxr-xr-x 1 user user 16K Jan 10 10:00 main_dynamic
```

The dynamically linked executable is **much smaller** because it doesn't contain the library code.

8.6.2 Startup Time Comparison

Static linking: Faster startup (no dynamic linking overhead)

Dynamic linking: Slower startup due to:

1. Loading and parsing the executable
2. Loading required shared libraries
3. Resolving symbols (at least for non-lazy-bound functions)

However, this difference is usually imperceptible for small programs.

8.6.3 Memory Usage When Running Multiple Programs

This is where dynamic libraries really shine:

Scenario: 10 programs using the same library
Static linking:

- Each program has its own copy of the library code in memory
- Total memory usage: $10\times$ library size

Dynamic linking:

- The library is loaded once into physical memory
- Each program has its own copy of the library's data (writable sections)
- The library's code (read-only sections) is shared among all processes
- Total memory usage: $1\times$ library code $+10\times$ library data

For large libraries like libc, the memory savings can be enormous.

8.7 Key Takeaways

1. **Static libraries (.a)** are archives of object files that are copied into executables at link time, producing self-contained but larger binaries.
2. **Dynamic libraries (.so)** are loaded at runtime by the dynamic linker, producing smaller binaries and enabling code sharing between processes.
3. **Position-independent code (PIC)** allows shared libraries to be loaded at any memory address, essential for dynamic linking.
4. **PLT and GOT** are data structures that enable runtime symbol resolution through indirection, with lazy binding deferring resolution until first use.
5. **Library search paths** (RPATH, LD_LIBRARY_PATH, system paths) control where the dynamic linker looks for shared libraries.
6. **Symbol visibility** controls which functions are exported from a library, enforcing API boundaries and improving load times.
7. **Tradeoffs**: Static linking gives simpler deployment and faster startup; dynamic linking gives smaller binaries and shared memory.

8.8 Looking Ahead

In Chapter 8, we'll explore the C standard library (libc) and system calls in depth. We'll see how functions like `printf` and `malloc` are implemented, and understand the boundary between user-space code and kernel services.

Chapter 9

Standard Library and System Calls

9.1 Introduction

Every C programmer uses functions like `printf`, `malloc`, `open`, and `read`. But where do these functions come from? Are they part of the C language, part of the operating system, or something else entirely?

This chapter explores the layered architecture that sits between your C code and the bare hardware. We'll distinguish between:

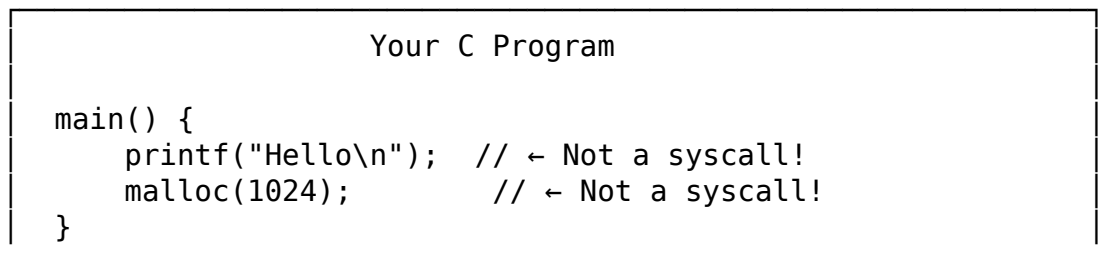
- **The C standard library (libc)**: User-space code that provides the standard C API
- **System calls**: The kernel interface that performs privileged operations
- **The boundary between user mode and kernel mode**: Where protection and privilege transitions occur

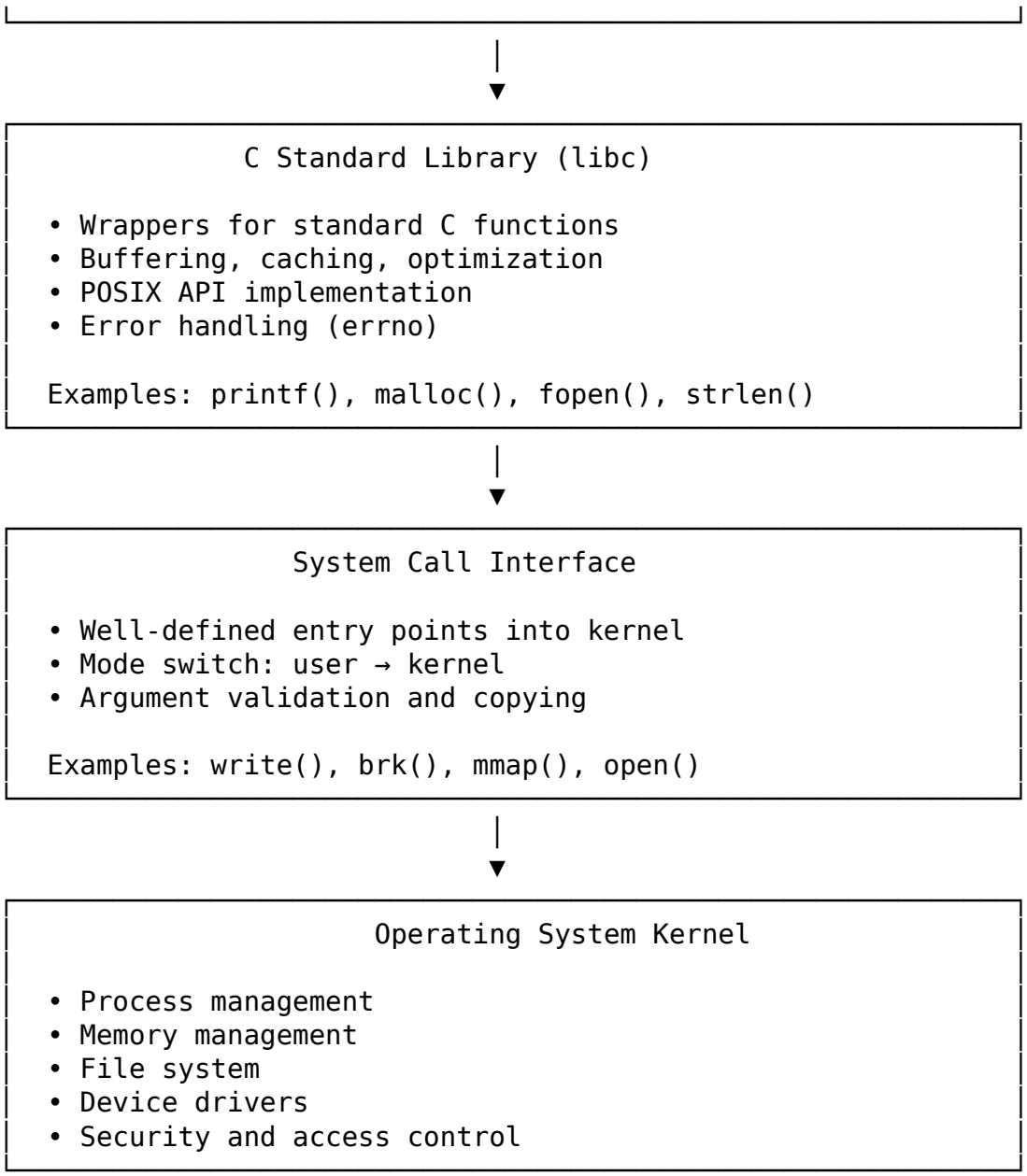
Understanding this distinction is crucial for:

- **Performance optimization**: Knowing when library functions vs. raw syscalls are appropriate
- **Debugging**: Understanding error reporting and stack traces
- **Portability**: Writing code that works across different platforms
- **Security**: Understanding privilege boundaries and potential vulnerabilities

9.2 The Layered Architecture

Modern Unix-like systems have a clear separation between user-space programs and the kernel:





Key insight: Most C library functions are **not** system calls. They're user-space code that may or may not eventually make system calls.

9.3 What is a System Call?

A **system call** (syscall) is a programmatic way a program requests a service from the operating system kernel. System calls are the only legitimate way to access kernel resources.

9.3.1 Why System Calls Exist

User programs cannot directly access:

- **Hardware devices**: Direct hardware access would allow any program to monopolize or damage devices
- **Other processes' memory**: Without isolation, buggy or malicious programs could corrupt other programs
- **Network stack**: Raw network access requires privilege and coordination
- **File system**: File permissions and access control need kernel enforcement

The kernel provides **controlled access** to these resources through system calls.

9.3.2 User Mode vs. Kernel Mode

CPUs support different privilege levels (called **rings** on x86):

- **Ring 0 (Kernel Mode)**: Unrestricted access to all hardware and memory
- **Ring 3 (User Mode)**: Restricted access; certain instructions cause traps

System calls provide a **controlled, secure gateway** from user mode to kernel mode.

9.3.3 How System Calls Work

Making a system call involves:

1. **Program preparation**: Place syscall number and arguments in registers
2. **Trap instruction**: Execute special instruction (e.g., `syscall` on x86-64)
3. **CPU mode switch**: CPU switches from user mode to kernel mode
4. **Kernel execution**: Kernel validates arguments and performs the operation
5. **Result return**: Kernel places result in a register and switches back to user mode

Example: The `write` syscall on x86-64

```
; User-space code making a raw syscall
mov rax, 1 ; Syscall number for 'write' (on x86-64 Linux)
mov rdi, 1 ; File descriptor 1 (stdout)
mov rsi, msg ; Pointer to message
mov rdx, 13 ; Length of message
syscall ; Invoke kernel

; Result is now in rax
```

System calls are the **only supported interface** between user and kernel code.

9.4 The C Standard Library (libc)

9.4.1 What is libc?

libc is the C standard library—a collection of functions that implement the C standard library (ISO C) and POSIX APIs. On Linux, the most common implementation is **glibc** (GNU C Library).

Key responsibilities:

- **Standard C functions**: `printf`, `malloc`, `strcpy`, etc.
- **System call wrappers**: `open`, `read`, `write`, etc.
- **Buffering and caching**: Optimize performance by batching operations
- **Thread safety**: Support for multithreaded programs
- **Locale support**: Internationalization and localization

9.4.2 libc as a Wrapper Layer

Most libc functions are **wrappers** around system calls, adding value through buffering, error handling, and abstraction.

Example 1: `printf`

`printf` is **not** a system call. It's a complex user-space function:

```
// Simplified view of what printf does
int printf(const char *format, ...) {
   va_list args;
   va_start(args, format);

   // 1. Format the string in user space
   char buffer[4096];
   int len = vsnprintf(buffer, sizeof(buffer), format, args);

   // 2. Write to stdout (which is buffered!)
   // stdout is a FILE* with its own buffer
   return fwrite(buffer, 1, len, stdout);

   // fwrite will eventually call the write() syscall,
   // possibly after buffering many small writes
}
```

The chain looks like:

```
printf()
    → vfprintf() [format the string]
        → fwrite() [manage stdio buffer]
            → write() syscall [actual kernel I/O]
```

Why this indirection?

1. **Performance**: Formatting in user space avoids kernel mode switches
2. **Buffering**: Multiple `printf` calls can be combined into one `write` syscall

3. **Flexibility**: The same formatting code works for files, strings, and custom streams

Example 2: `malloc`
`malloc` is also **not** a system call (mostly):

```
void *malloc(size_t size) {
   // Ask the heap allocator (brk/mmap syscalls under the hood)
   // The heap allocator manages a large memory region
   // and carves it into small pieces for your program

   // Most malloc() calls don't trigger syscalls!
   // They just carve out space from already-allocated regions
   return heap_alloc(size);
}
```

The heap allocator (`malloc` implementation) manages large memory regions obtained from the kernel via `brk` or `mmap` syscalls. Most `malloc` calls just carve out space from these regions—no syscall needed!

Example 3: `strlen`
`strlen` is purely user-space:

```
size_t strlen(const char *s) {
   size_t len = 0;
   while (s[len]) len++; // Just reads memory!
   return len;
}
```

No syscall involved—it's just reading your program's own memory.

9.4.3 When libc Does Make Syscalls

Some libc functions are thin wrappers around syscalls:

```
// glibc's open() function (simplified)
int open(const char *pathname, int flags, mode_t mode) {
   // This is basically just a syscall wrapper
   return syscall(__NR_open, pathname, flags, mode);
}
```

But even "simple" wrappers add value:

- **Error handling**: Convert syscall return values to `errno`
- **Thread safety**: Handle cancellation points for pthreads
- **Cancellation checking**: Allow threads to be cancelled at safe points

9.5 System Call Numbers and Conventions

9.5.1 Syscall Numbers

Each system call has a unique number. On Linux x86-64:

```
0   read
1   write
2   open
3   close
... hundreds more ...
```

These numbers are architecture-specific! ARM64 has different numbers than x86-64.

Raw syscall example:

```
// Make a write syscall directly (bypassing libc)
#include <unistd.h>
#include <sys/syscall.h>

int main(void) {
   const char msg[] = "Hello␣via␣raw␣syscall!\n";

   // Direct syscall (x86-64 Linux)
   syscall(__NR_write, 1, msg, sizeof(msg) - 1);

   return 0;
}
```

9.5.2 Syscall Calling Conventions

On x86-64 Linux:

- **Syscall number**: RAX register
- **Arguments**: RDI, RSI, RDX, R10, R8, R9 (in order)
- **Return value**: RAX register
- **Error indication**: Return value between -4095 and -1 (negated errno)

Note the difference from function calls:

- Function calls use RCX for 4th argument
- Syscalls use R10 for 4th argument (syscall instruction clobbers RCX)

9.6 Error Reporting: errno

9.6.1 How errno Works

When a syscall fails, the kernel returns a negative error code. libc converts this to:

1. **Return value**: -1 (for most functions)
2. **errno**: A global variable containing the error code

```c
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main(void) {
   FILE *f = fopen("/nonexistent/file", "r");

   if (f == NULL) {
      // errno is set by libc based on kernel error
      printf("Error %d: %s\n", errno, strerror(errno));
   }

   return 0;
}
```

Important: `errno` is thread-local! Each thread has its own copy, so multithreaded programs work correctly.

9.6.2 Why errno Exists

Why not just return the error code directly?

1. **Historical**: Early C had limited error handling options
2. **Efficiency**: Negative return values can indicate valid data (e.g., `read` can legitimately return 0 for EOF)
3. **Detail**: errno provides more error information than just ”success/failure”

9.7 Observing Syscalls with strace

`strace` is a powerful tool that traces system calls made by a program.

9.7.1 Basic strace Usage

Create `simple.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
   char *s = strdup("Hello");
   printf("%s\n", s);
   free(s);
   return 0;
}
```

Compile and trace:

```
gcc -o simple simple.c
strace ./simple
```

Typical output (abbreviated):

```
execve("./simple", ["./simple"], 0x7ffc...) = 0
brk(NULL)                               = 0x55555000
brk(0x55556c000)                        = 0x55556c000
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7ffff
write(1, "Hello\n", 6)                  = 6
munmap(0x7ffff..., 4096)                = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

What to observe:

- `brk`/`mmap`: Memory allocation for heap
- `write`: The actual I/O syscall (only one, despite printf!)
- No syscall for `strlen`-like operations (they're pure user-space)

9.8 Library vs. Syscall Performance

9.8.1 The Cost of Syscalls

System calls are expensive compared to function calls:

```
Function call: ~1-5 nanoseconds
System call: ~500-2000 nanoseconds
```

The overhead comes from:

1. **Mode switch**: User → kernel → user transition
2. **Validation**: Kernel must validate all arguments
3. **Scheduling**: Other processes might run while in kernel
4. **Cache effects**: Kernel code uses different cache lines

9.8.2 libc Buffering

libc performs buffering at several levels:

Stdio buffering (`FILE*` streams):

```c
// Unbuffered (slow)
setbuf(stdout, NULL);
for (int i = 0; i < 1000; i++) {
   printf("x"); // Syscall each iteration!
}

// Fully buffered (fast)
setvbuf(stdout, NULL, _IOFBF, BUFSIZ);
for (int i = 0; i < 1000; i++) {
   printf("x"); // Buffered until:
```

```
            // - Buffer is full
            // - fflush() is called
            // - Program exits
}
```

Default buffering behavior:

- **stdin**: Line-buffered (when interactive) or fully buffered
- **stdout**: Line-buffered (when connected to terminal) or fully buffered
- **stderr**: Always unbuffered (ensure error messages appear)

9.9 Key Takeaways

1. **libc vs. kernel**: Most C library functions are user-space code that may eventually trigger syscalls, but many operations (like strlen, malloc from existing heap, etc.) don't require kernel interaction.
2. **Syscalls are expensive**: System calls cost ~500-2000 nanoseconds, much more than function calls. This is why libc buffering is so important for performance.
3. **Buffering matters**: libc's stdio buffering can reduce syscall count by orders of magnitude, dramatically improving performance for I/O operations.
4. **strace is powerful**: The strace tool reveals exactly which syscalls a program makes, invaluable for debugging and performance analysis.
5. **Error reporting**: The kernel returns error codes in registers; libc converts these to errno for easier error handling.
6. **Memory management**: malloc/free mostly manage user-space heap, only calling brk/mmap when more memory is needed from the kernel.
7. **User/kernel boundary**: Understanding this boundary is essential for writing efficient, portable C programs and for debugging system-level issues.

9.10 Looking Ahead

In Chapter 9, we'll explore **ABI and cross-platform compilation**—how different architectures and operating systems define binary interfaces, and what it takes to make code portable across different platforms.

Chapter 10

ABI and Cross-Platform Compilation

10.1 Introduction

We've seen how C code is compiled, linked, and executed. But what happens when you need to run your compiled binary on a different machine with a different CPU architecture or operating system? Why can't you just copy the binary and run it?

This chapter explores **ABI (Application Binary Interface)**—the set of conventions that enable binary code to work together at the machine level. Unlike API (source-level compatibility), ABI is about binary compatibility: how functions are called, how data is laid out in memory, and how the operating system expects programs to behave.

Understanding ABI is crucial for:

- **Cross-platform development**: Writing code that compiles and runs correctly on different platforms
- **Cross-compilation**: Building binaries for one architecture on another
- **Library development**: Creating libraries that work across different compilers and platforms
- **Debugging**: Understanding mysterious crashes when mixing code from different sources

10.2 What is an ABI?

An **Application Binary Interface (ABI)** is a contract between compiled code components that specifies:

1. **Calling convention**: How functions call each other (argument passing, return values, stack cleanup)
2. **Data representation**: Size and alignment of data types
3. **Register usage**: Which registers are preserved vs. volatile
4. **Object file format**: ELF, PE, Mach-O, etc.
5. **System call interface**: How to request kernel services

6. **Runtime behavior**: Stack layout, exception handling, initialization

ABI vs. API:

API (Application Programming Interface) Source-level interface. Defines function names, parameters, return types. Enables recompilation with different compilers/languages. Example: C header files.

ABI (Application Binary Interface) Binary-level interface. Defines binary representation and calling conventions. Enables linking code compiled by different compilers. Example: System V AMD64 ABI.

Tip

API compatibility means you can recompile the code. ABI compatibility means you can link the binaries directly without recompilation.

10.3 Components of an ABI

10.3.1 Data Type Representation

ABIs specify the size and representation of fundamental data types.

Example: x86-64 Linux (LP64)

Type	Size	Alignment	Notes
`char`	1 byte	1	-
`short`	2 bytes	2	-
`int`	4 bytes	4	-
`long`	8 bytes	8	Longs are 64-bit!
`long long`	8 bytes	8	-
`void*`	8 bytes	8	Pointers are 64-bit
`float`	4 bytes	4	IEEE 754 single
`double`	8 bytes	8	IEEE 754 double

Naming schemes:

- **ILP32**: int, long, pointer are 32-bit
- **LP64**: long and pointer are 64-bit (int is 32-bit)
- **LLP64**: long long and pointer are 64-bit (Windows x64)

10.3.2 Calling Conventions

ABIs define how functions call each other—where arguments go, where return values go, who cleans the stack.

System V AMD64 ABI (Linux/Unix x86-64):
Arguments (in order):

1. RDI, RSI, RDX, RCX, R8, R9 (first 6 integer/pointer arguments)
2. XMM0-XMM7 (first 8 floating-point arguments)

3. Stack (remaining arguments)

Return value:

- Integer/pointer: RAX
- Floating-point: XMM0
- Large structs: Hidden pointer parameter

Microsoft x64 ABI (Windows x64):
Arguments (in order):

1. RCX, RDX, R8, R9 (first 4 arguments)
2. Stack (remaining arguments)

Return value:

- RAX (integer/pointer)
- XMM0 (floating-point)

Key difference: Different registers for arguments! This is why you can't link object files between Windows and Linux.

10.4 Cross-Compilation

10.4.1 What is Cross-Compilation?

Native compilation: Build binaries for the same architecture/host

- x86-64 Linux → x86-64 Linux binary

Cross-compilation: Build binaries for a different architecture/host

- x86-64 Linux → ARM64 Linux binary
- x86-64 macOS → x86-64 Linux binary

10.4.2 Cross-Compilation Toolchains

A cross-compilation toolchain includes:

- **Cross-compiler**: gcc that generates code for target architecture
- **Cross-assembler**: as for target
- **Cross-linker**: ld for target
- **Sysroot**: Target system's headers and libraries

Naming convention: `{host}-{target}-{tool}`
Examples:

- `x86_64-linux-gnu-gcc`: Native compiler for x86-64 Linux
- `aarch64-linux-gnu-gcc`: Cross-compiler for ARM64 Linux (from x86-64 host)
- `x86_64-w64-mingw32-gcc`: Cross-compiler for Windows (from Linux host)

10.4.3 Installing Cross-Compilers

Ubuntu/Debian:

```
# ARM64 cross-compiler
sudo apt-get install gcc-aarch64-linux-gnu

# Windows cross-compiler
sudo apt-get install gcc-mingw-w64-x86-64

# Check installation
aarch64-linux-gnu-gcc --version
x86_64-w64-mingw32-gcc --version
```

10.4.4 Cross-Compiling a Program

Simple cross-compilation:

```
# Compile for ARM64 instead of x86-64
aarch64-linux-gnu-gcc -o hello_arm64 hello.c

# Check the output file
file hello_arm64
# Output: ELF 64-bit LSB executable, ARM aarch64...

# It won't run on x86-64!
./hello_arm64
# bash: ./hello_arm64: cannot execute binary file

# But it works on ARM64 systems
scp hello_arm64 user@arm64-machine:~
```

10.5 ELF vs. PE vs. Mach-O

Different operating systems use different executable file formats:

10.5.1 ELF (Executable and Linkable Format)

Used by: Linux, most Unix-like systems, many embedded systems
File type: `ELF 64-bit LSB executable, x86-64, version 1 (SYSV)`

10.5.2 PE (Portable Executable)

Used by: Windows
File type: `PE32+ executable (console) x86-64, for MS Windows`

10.5.3 Mach-O

Used by: macOS, iOS
File type: `Mach-O 64-bit executable x86_64`

10.6 Key Takeaways

1. **ABI is binary-level contract**: Unlike API (source-level), ABI defines how binaries interact—data layout, calling conventions, register usage.
2. **Data types vary by platform**: `long`, `void*`, and even `int` may have different sizes. Use fixed-width types (`int32_t`) for binary data.
3. **Calling conventions differ**: x86-64 System V (Linux) uses different argument registers than Microsoft x64 (Windows), preventing binary compatibility.
4. **Cross-compilation requires toolchains**: Build binaries for different architectures using cross-compilers like `aarch64-linux-gnu-gcc`.
5. **Executable formats differ**: ELF (Linux), PE (Windows), and Mach-O (macOS) are incompatible at the binary level.

10.7 Looking Ahead

In Chapter 10, we'll explore **process loading and memory layout**—how the operating system loads executables into memory and sets up the runtime environment.

Chapter 11

Process Loading and Memory Layout

11.1 Introduction

When you type `./myprogram` and press enter, what actually happens? How does a simple executable file on disk become a running process with memory, stack, and heap? This chapter explores the final piece of the compilation story: **process loading** and **memory layout**.

We've seen how source code becomes an executable file. Now we'll see how that executable file is brought to life by the operating system's loader. We'll examine:

- **The loader**: How the OS reads and maps executable files
- **Virtual memory**: How each process gets its own address space
- **Memory segments**: text, data, bss, heap, stack, and more
- **Dynamic linking**: How shared libraries are loaded at runtime
- **Memory mapping**: How `mmap` enables efficient file I/O and memory allocation

Understanding process loading is crucial for:

- **Debugging**: Understanding segmentation faults, memory errors
- **Performance**: Optimizing memory usage and allocation strategies
- **Security**: Understanding ASLR, stack protection, and memory isolation
- **Systems programming**: Writing low-level code that interacts with the OS

11.2 From Executable to Process

11.2.1 What is a Process?

A **process** is an instance of a program in execution. Unlike a program (static file on disk), a process is dynamic:

```
Program (File on Disk)          Process (Running Instance)
├── Code sections               ├── Memory regions
├── Data sections               ├── Stack
├── Metadata (headers, etc.)    ├── Heap
```

```
└── ...                          ├── Program counter
                                 ├── Open files
                                 └── ...
```

Key differences:

- **Program**: Static, passive, stored on disk
- **Process**: Dynamic, active, lives in memory

11.2.2 The Loading Process

When you run a program, the OS performs these steps:

```
1. User executes: ./myprogram
         │
         ▼
2. Shell calls fork() + execve()
         │
         ▼
3. Kernel's execve() handler:
   a. Read ELF header from file
   b. Verify it's a valid executable
   c. Create new memory map for process
   d. Read program headers (PHDRs)
   e. Map segments into memory (mmap)
   f. Set up stack and arguments
   g. Set up dynamic linker (if needed)
   h. Jump to entry point (_start)
         │
         ▼
4. _start (in libc/crt0):
   a. Initialize libc
   b. Run constructors (.init_array)
   c. Call main()
   │
   ▼
5. Your program runs!
   │
   ▼
6. After main() returns:
   a. Run destructors (.fini_array)
   b. Call exit()
   │
   ▼
7. Kernel cleans up process
```

11.3 Virtual Memory

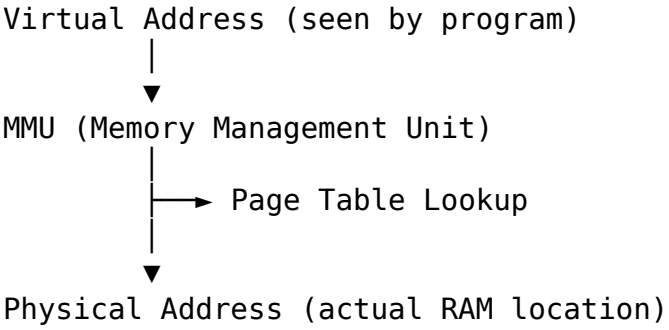
11.3.1 What is Virtual Memory?

Virtual memory gives each process the illusion of having its own private, contiguous address space. In reality, physical memory is fragmented and shared among all processes.

With virtual memory:

- Each process has its own address space (0 to $2^{64} - 1$ on 64-bit)
- The Memory Management Unit (MMU) translates virtual $\rightarrow$ physical addresses
- Processes are isolated from each other
- Pages can be moved, swapped, or shared transparently

Translation process:



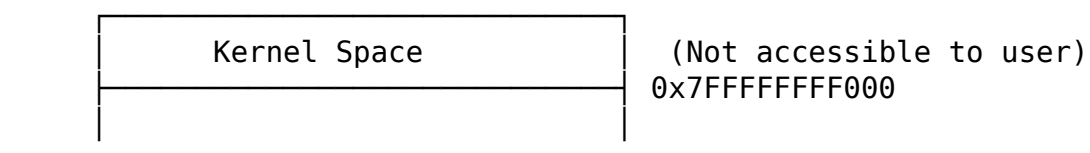
11.3.2 Benefits of Virtual Memory

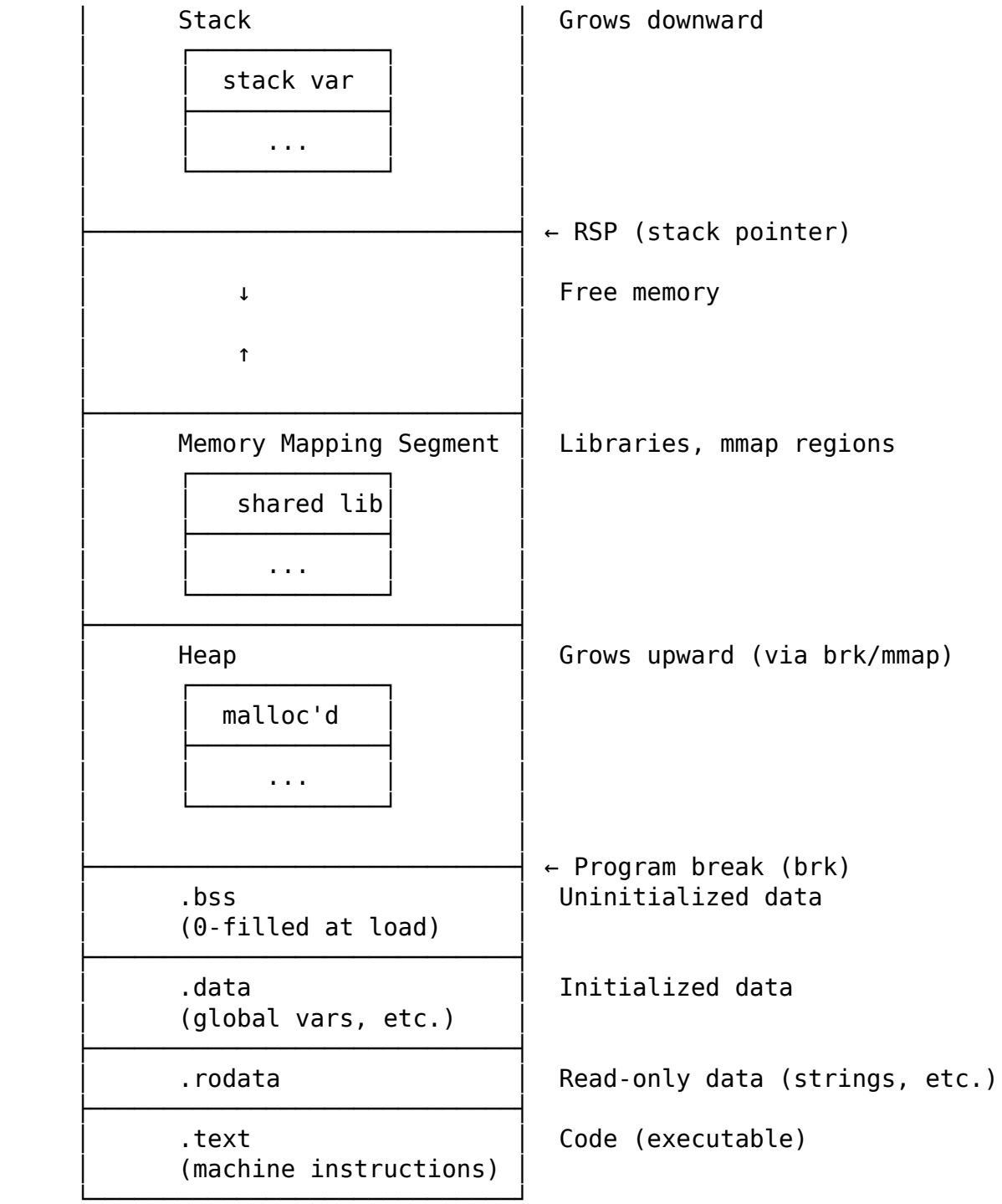
1. **Isolation**: Processes can't access each other's memory
2. **Protection**: Pages can be marked read-only, execute-only, etc.
3. **Efficiency**: Only loaded portions of a program need to be in RAM
4. **Simplicity**: Each program can use the same address range
5. **Sharing**: Multiple processes can share the same physical page (for libraries)

11.4 Memory Segments

11.4.1 Typical Process Memory Layout

A typical process memory layout on x86-64 Linux:





11.4.2 The .text Segment

Contains executable code (machine instructions).

Properties:

- **Read-only**: Code cannot modify itself

- **Executable**: CPU can fetch and execute instructions
- **Shared**: Multiple instances of the same program can share this segment

11.4.3 The .data Segment

Contains **initialized** global and static variables.
Properties:

- **Read-write**: Variables can be modified
- **Initialized**: Contains initial values from executable
- **Fixed size**: Size known at compile time

11.4.4 The .bss Segment

Contains **uninitialized** global and static variables.
Properties:

- **Read-write**: Variables can be modified
- **Zero-initialized**: Set to 0 at process start
- **Not stored in executable**: Only size is recorded, not data

Why separate .bss from .data?
Efficiency! .bss doesn't take up space in the executable file:

```
.data in file:
[initial values for 100 bytes] → 100 bytes in executable

.bss in file:
[size: 1000, zero at load] → 8 bytes in executable (just the size!)
```

The loader allocates .bss memory and zeros it at runtime.

11.4.5 The Heap

Dynamic memory allocation region managed by `malloc`/`free`.
Growth direction: Upward (toward higher addresses)
Management:

- libc's heap allocator (ptmalloc, jemalloc, etc.) manages this region
- Requests more memory from kernel via `brk()` or `mmap()` when needed
- Carves large regions into small allocations for your program

11.4.6 The Stack

Function call region for local variables, return addresses, saved registers.
Growth direction: Downward (toward lower addresses)
Stack overflow: Exceeding stack size causes segmentation fault

11.5 The Loader in Detail

11.5.1 Program Headers

ELF executables contain **program headers** that tell the loader how to load the file.

Key program headers:

- **PHDR**: Program header table itself
- **INTERP**: Specifies dynamic linker (ld-linux.so)
- **LOAD**: Loadable segment (code or data)
- **DYNAMIC**: Dynamic linking information
- **GNU_STACK**: Stack permissions (executable? non-executable?)

11.5.2 The Dynamic Linker

For dynamically linked executables, the **dynamic linker** (`ld-linux.so`) performs final setup:

What it does:

1. Load required shared libraries (libc.so, etc.)
2. Resolve symbol references (PLT/GOT, as seen in Chapter 7)
3. Run relocations
4. Call shared library initializers
5. Transfer control to the program

11.5.3 ASLR: Address Space Layout Randomization

Modern systems use **ASLR** to randomize memory locations for security:

Without ASLR (old systems):

```
Stack always at 0x7fffff00000
Heap always starts at 0x02000000
Libraries always loaded at same addresses
→ Predictable = easier to exploit!
```

With ASLR (modern systems):

```
Stack at random location each run
Heap starts at random location
Libraries loaded at random addresses
→ Unpredictable = harder to exploit!
```

11.6 Key Takeaways

1. **Loading transforms executables into processes**: The OS loader reads the ELF file, maps segments into memory, and transfers control to the entry point.
2. **Virtual memory isolates processes**: Each process has its own address space, translated to physical memory by the MMU via page tables.
3. **Memory is organized into segments**: .text (code), .rodata (read-only data), .data (initialized data), .bss (uninitialized data), heap, and stack.
4. **The stack grows downward, heap grows upward**: Local variables and function frames are on the stack; dynamic allocations are on the heap.
5. **The loader uses program headers**: ELF program headers specify which segments to load, their permissions, and their virtual addresses.
6. **Dynamic linking happens at load time**: The dynamic linker loads shared libraries, resolves symbols, and performs relocations before your code runs.
7. **ASLR randomizes memory layout**: For security, ASLR randomizes stack, heap, and library locations to make exploitation harder.

11.7 Looking Ahead

This completes our journey from C source code to running process! We've seen:

- **Chapter 1-2**: Compilation pipeline and preprocessing
- **Chapter 3-4**: Compilation internals and object files
- **Chapter 5-6**: ELF format and linking
- **Chapter 7-8**: Static/dynamic libraries and system calls
- **Chapter 9**: ABI and cross-platform compilation
- **Chapter 10**: Process loading and memory layout

You now understand the complete path from source code to executing process, with deep knowledge of each step along the way.